ITCS213 Computer Organization and Architecture

Class: 1-2026
Sec 2 Fri 9:00-12:00 IT312 (Aj. Joke)
Sec 3 Fri 13:00-16:00 IT312 (Aj. Joke)
Sec 1 Fri 13:00-16:00 IT311 (Aj. Pete)

Unit 8: **Datapath, I-Type Instructions**, **Multiplication**

Lecturer: Dr. Peter M.D. Scully
Faculty of ICT, Mahidol University
peter.scu@mahidol.edu

Slides adapted from Subject Coordinator:
Dr. Kanoksak Wattanachote (Ph.D) (金功勳 博士)
kanoksak.wat@mahidol.ac.th

Announcement: ITCS213

Unit 8: Datapath, I-Type Instructions, Multiplication

Announcements

- Feedback & responses.
- In-class Quizzes
- **Midterm exam** on **FinEx**
  - 1-page of handwritten notes.
  - **2-hours only.**
  - “Open book” *(not much time)*
    (MyCourses slides available)
- Next class is **Fri Oct 9th**.

Agenda

- Reminder for sll and add for binary multiplication.
- **Walkthrough of R-Type Datapath**
  MIPS ISA 4-Stage Pipeline of a 32-bit instruction.
- I-Type Instruction Format and Immediate Constants Example li
- Pseudo Instructions: move, not, neg, li
- **Integer Multiplication in MIPS with:**
  - **mult**, **multu**, **div**, **divu**,
  - **mfhi**, **mflo** for **HI** / **LO** registers.
  - **mul** special case mul.

Grading

ITCS213 Computer Organization and Architecture

- Overall Score Parts
  - Class activities + Assignments + Quizzes 20%
  - Midterm Exam 40%
  - Final Exam 40%

GRADING SYSTEM				
Grade Criteria		A	>=	80
		B+	>=	75
		B	>=	70
		C+	>=	65
		C	>=	60
		D+	>=	55
		D	>=	50
		F	<	50

PART A

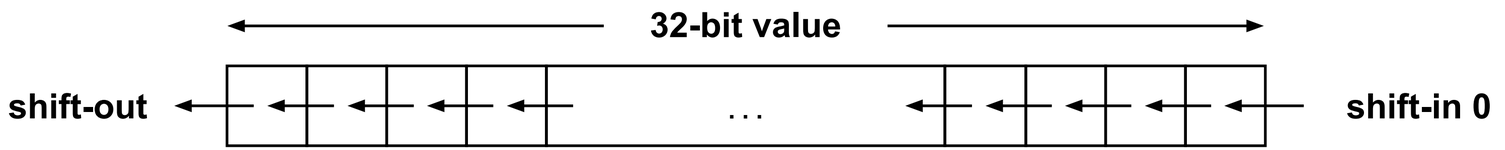
Binary Multiplication

with sll & add

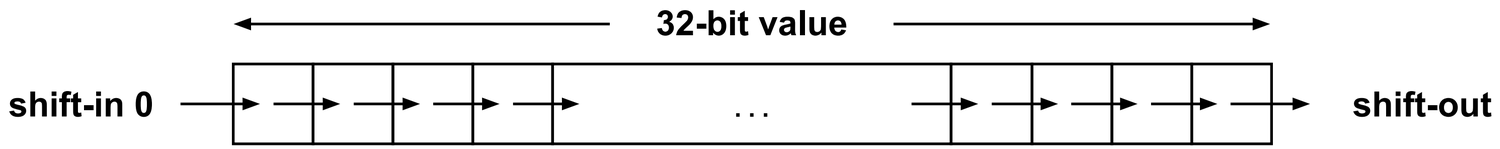
Multiplying in binary: the shift-and-add method.

Shift Operations

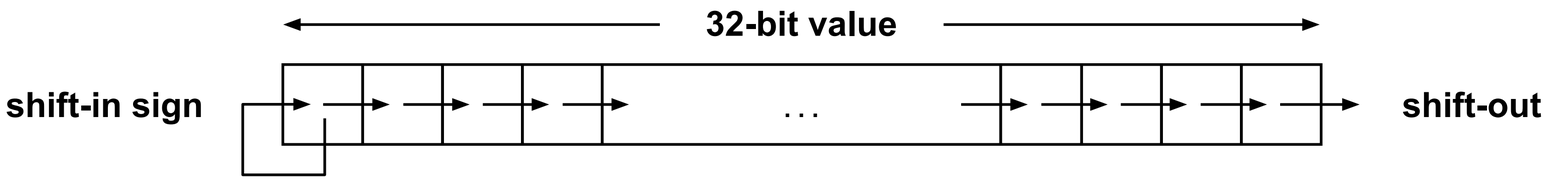
- Shifting is the process of moving the 32 bits of a number left or right.
- These instructions use the **5-bit shift amount (sa)** field.
- **`sll`** means **shift left logical** (insert zero from the right).



- **`srl`** means **shift right logical** (insert zero from the left).



- **`sra`** means **shift right arithmetic** (insert sign-bit).



Binary Multiplication

- **Shift Left Instruction** (**sll**) can perform multiplication
  - When the **multiplier** is a power of 2
- You can factor any binary number into powers of 2
- Example: multiply **$t0** by **36**

```
$t0*36 = $t0*(4 + 32) = $t0*4 + $t0*32
   sll  $t1, $t0, 2    # $t1 = $t0 * 4
   sll  $t2, $t0, 5    # $t2 = $t0 * 32
   addu $t3, $t1, $t2  # $t3 = $t0 * 36
```

- 5 in binary: 0000 0101
- Shift left 1 bit: 0000 1010 = 10 (5×2)
- Shift left 2 bits: 0001 0100 = 20 (5×4)
- Shift left 3 bits: 0010 1000 = 40 (5×8)

Exercise

Binary Multiplication

- Multiply **$t0** by **26**, using shift and add instructions

$t0*26 = $t0*(_____) = _____

```
sll  $__, $__, __      # $__ = $__ * __
sll  ...               # ...
sll  ...               # ...
addu $__, $__, $__     # $__ = $__ * __
addu ...               # ...
```



Exercise

Binary Multiplication

- Multiply **$t0** by **31**, using shift and add instructions

$t0*31 = $t0*(_____) = _____

```
sll  $__, $__, __      # $__ = $__ * __
sll  ...               # ...
sll  ...               # ...
addu $__, $__, $__     # $__ = $__ * __
addu ...               # ...
```





Key Points: Binary Multiplication Review

- **Shift left (sll) multiplies by powers of 2**
  - sll by n bits multiplies value by 2^n
- **Multiply by 26: decompose as 16+8+2, shift and add**
  - Three shifts (by 4, 3, 1) combined with addition
- **Multiply by 31: decompose as 32-1, shift and subtract**
  - One shift (by 5) then subtract original value

PART C

Integer Multiplication & Division in MIPS

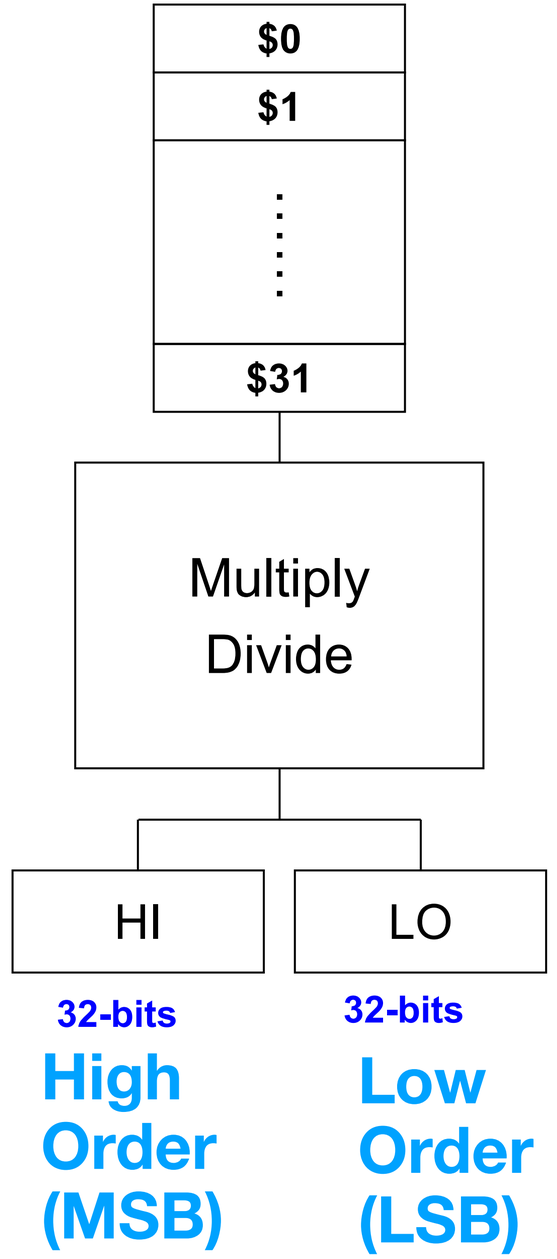
Multiplying and dividing integers with the HI and LO registers.

Integer Multiplication in MIPS

- Multiply instructions
  mult $s1,$s2 #Signed multiplication
  multu $s1,$s2 #Unsigned multiplication

- 32-bit multiplication produces a **64-bit Product**.
  - HI = **high-order** 32-bit of product.
  - LO = **low-order** 32-bit of product.

- Moving data from HI/LO to MIPS registers.
  mfhi Rd #move from HI to Rd
  mflo Rd #move from LO to Rd

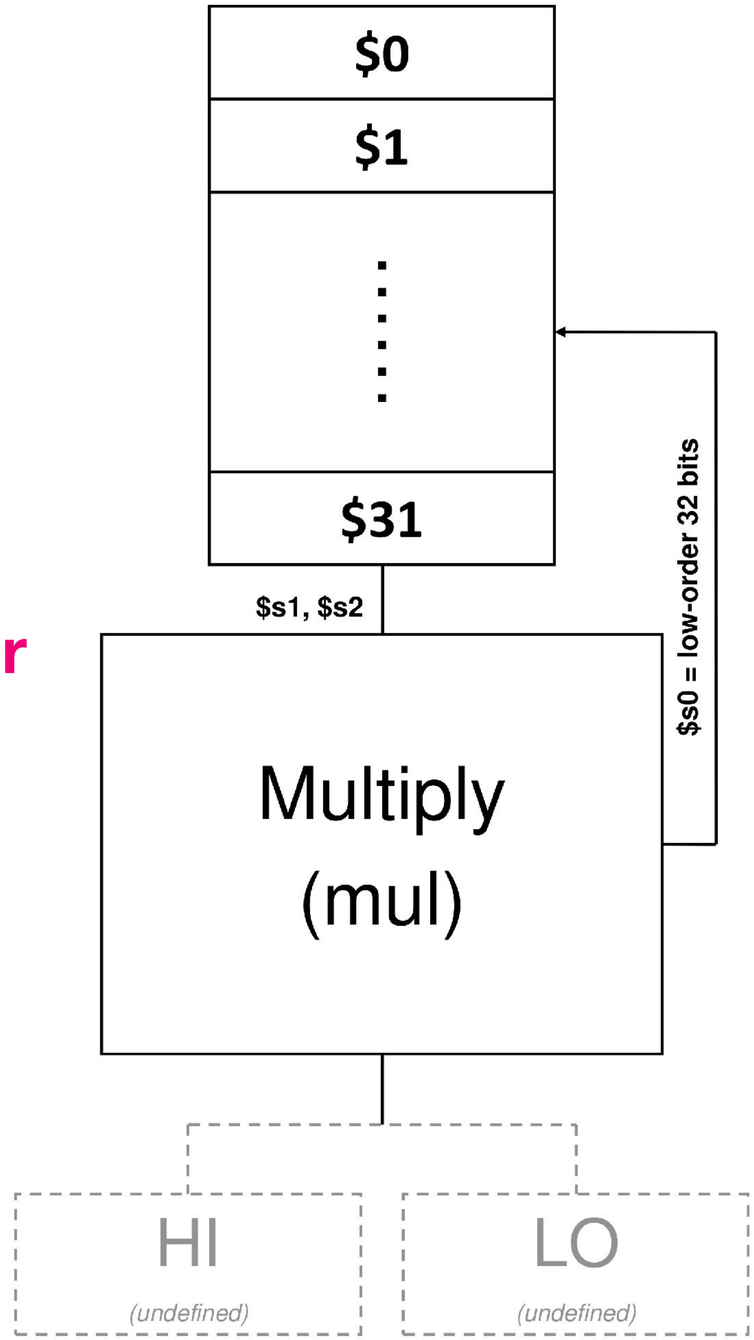


Integer Multiplication in MIPS

- MIPS also has a special **mul instruction**
  mul $s0,$s1,$s2 #$s0 = $s1 × $s2

  Put the **low-order 32 bits** into the **destination register (Rd)**.

- HI & LO are undefined.



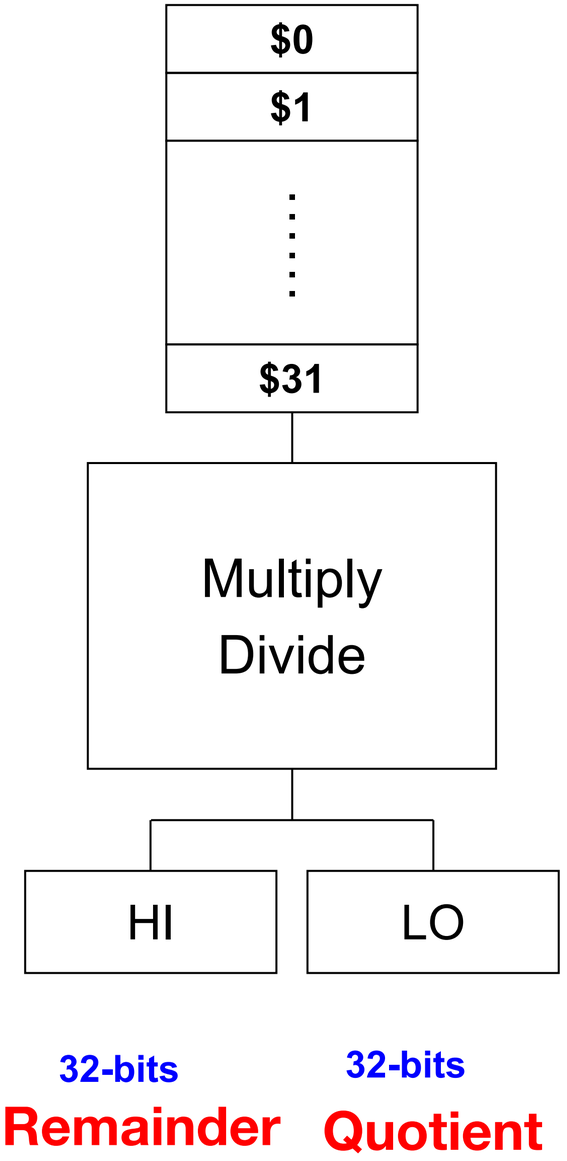
Integer Division in MIPS

- Divide instructions
  div $s1,$s2 #Signed division
  divu $s1,$s2 #Unsigned division

- Division produces a Quotient and a Remainder.
  - HI = 32-bit **Remainder**
  - LO = 32-bit **Quotient**

- Moving data from HI/LO to MIPS registers.
  mfhi Rd #move from HI to Rd
  mflo Rd #move from LO to Rd

If the **divisor is 0**, then the result is **unpredictable**.



Multiplication & Division Instructions

Instruction	Meaning	Op	Rs	Rt	Rd	sa	func
mult Rs, Rt	Hi, Lo = Rs × Rt	0	Rs	Rt	0	0	0x18
multu Rs, Rt	Hi, Lo = Rs × Rt	0	Rs	Rt	0	0	0x19
mul Rd, Rs, Rt	Rd = Rs × Rt	1C	Rs	Rt	Rd	0	0x02
div Rs, Rt	Hi, Lo = Rs / Rt	0	Rs	Rt	0	0	0x1A
divu Rs, Rt	Hi, Lo = Rs / Rt	0	Rs	Rt	0	0	0x1B
mfhi Rd	Rd = Hi	0	0	0	Rd	0	0x10
mflo Rd	Rd = Lo	0	0	0	Rd	0	0x12

- **Signed** arithmetic: `mult`, `div` (`Rs` and `Rt` are signed).
- **Unsigned** arithmetic: `multu`, `divu` (`Rs` and `Rt` are unsigned).
- **NO arithmetic exception can occur.**



Key Points: Integer Multiplication & Division in MIPS

- **mult/multu: 64-bit product stored in Hi:Lo register pair**
  - Signed (mult) vs unsigned (multu) multiplication
- **div/divu: Lo = quotient, Hi = remainder**
  - Signed (div) vs unsigned (divu) division
- **mfhi and mflo retrieve results from Hi/Lo registers**

PART B

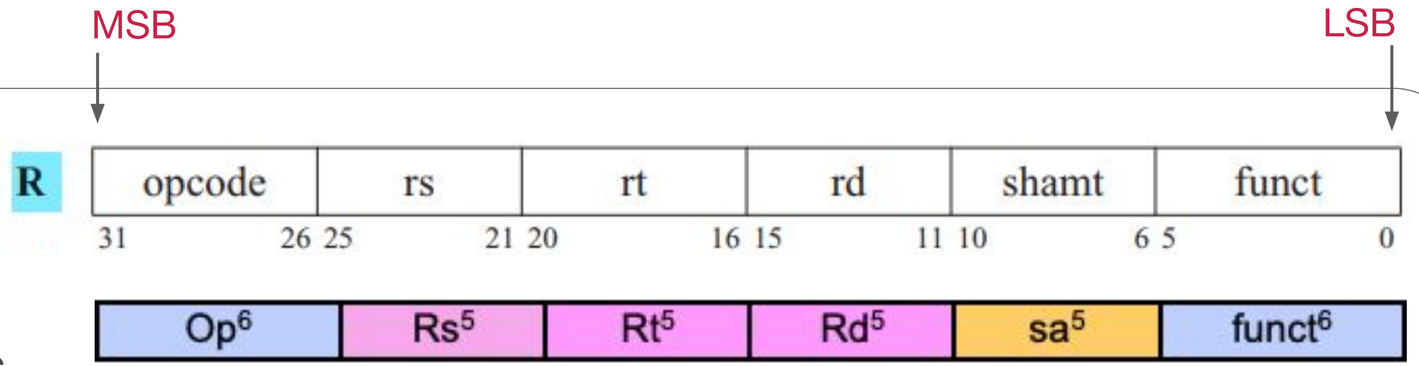
Walkthrough of R-Type Datapath

(4-Stage Pipeline)

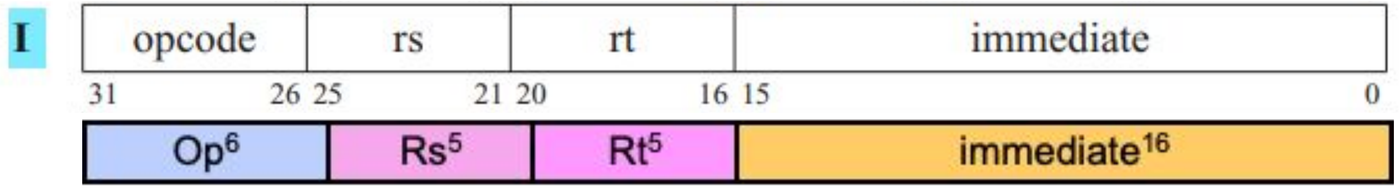
Instruction Formats

- All instructions are 32-bit wide.

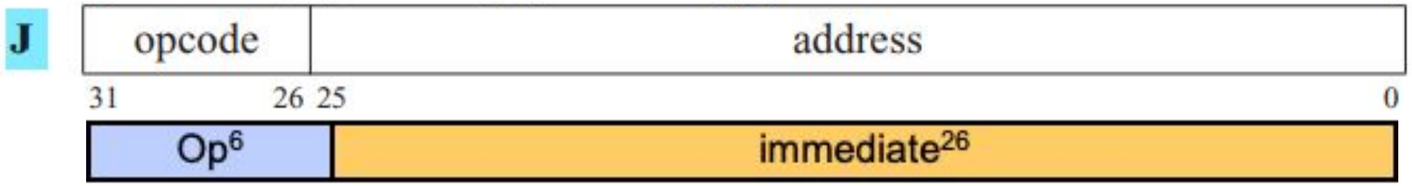
- **Register** (R-Type)
  - **Register**-to-register instructions.
  - Op: operation code specifies the format of the instruction.



- Immediate (**I-Type**)
  - A 16-bit immediate constant is part of the instruction.

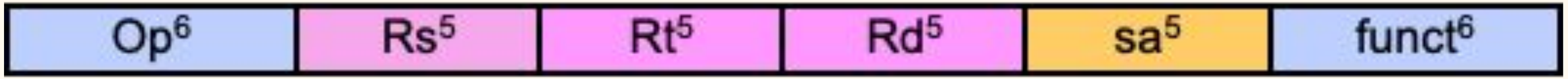


- Jump (J-Type)
  - Used by jump instructions.



Instruction Formats

Exercise



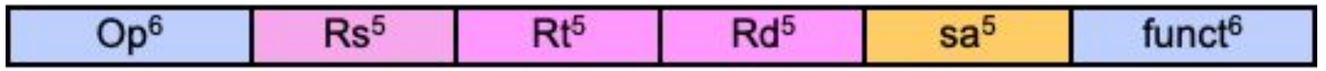
1. Which instruction type is it?

2. Is the 0th bit on the left or right?
   (The smallest digit)



Walkthrough of R-Type Datapath

- The example: **`funct`** code 100000 means **`add`**, 100100 means **`and`**.
- The other fields are:
  - **Rs**: First source **register**
  - **Rt**: Second source **register**
  - **Rd**: Destination **register** (where the result goes)
  - **sa** or **shamt**: **Shift amount** *(used for shift instructions like **sll**, otherwise 0)*
  - **funct**: Function code (specifies the exact operation)



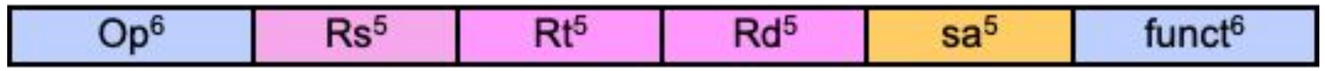
Walkthrough of R-Type Datapath

The key to understanding the instructions is to familiarize yourself with their format.

All MIPS instructions are 32 bits long, but are divided differently.

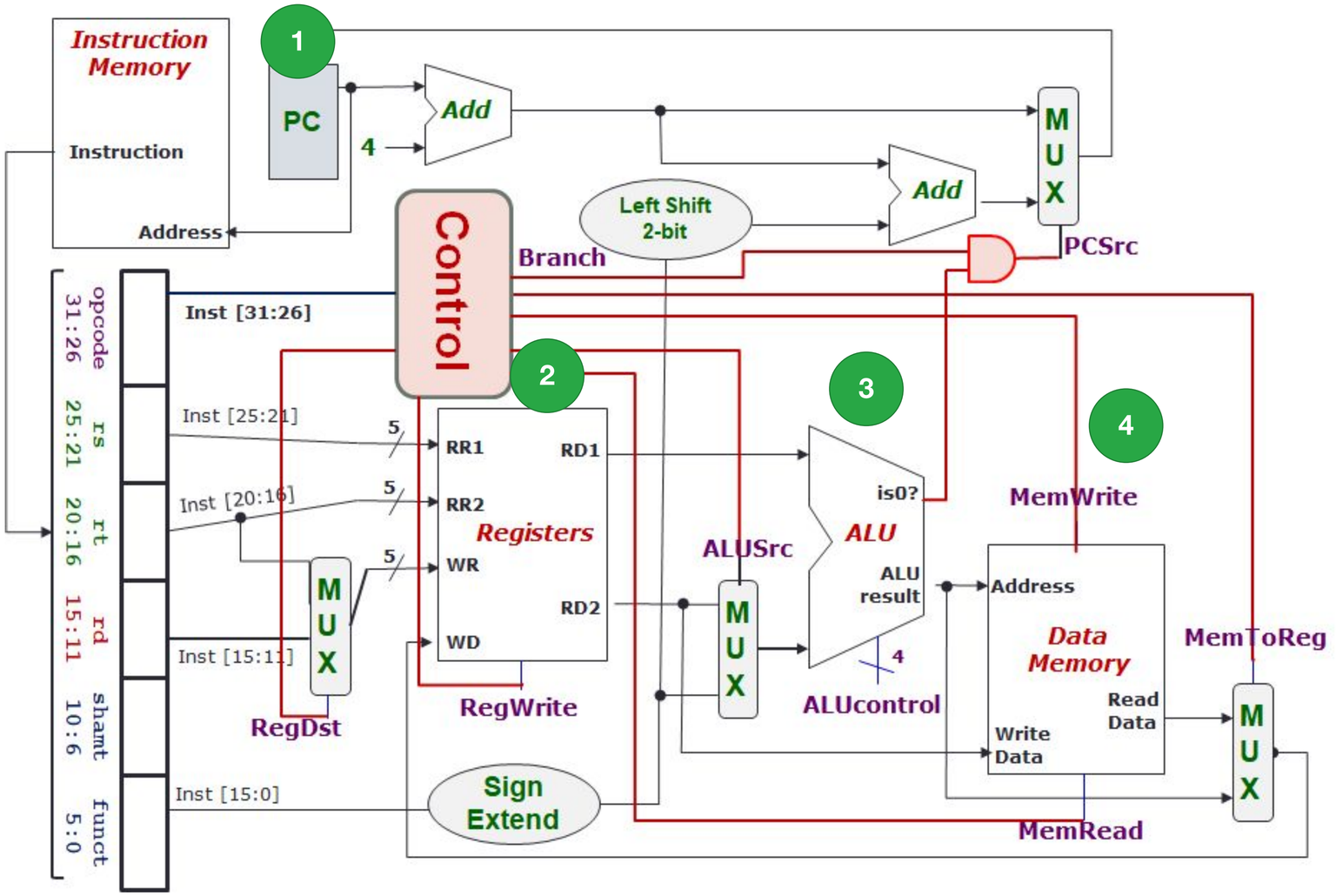
- R-Type (**Register**-Type)
  - Purpose: For operations where all operands are registers, e.g.,

    add $t1, $t2, $t3
  - Format:



  - **Why Op is often 0**:
    - The **Op** field (the first 6 bits) is the operation code.
    - For most basic R-type instructions (**`add`**, **`sub`**, **`and`**, **`or`**, etc.), **the Op code is 000000** — this tells the CPU:
      - "Hey, this is an R-type **instruction** — to find out which R-type **instruction**, **look at the very last 6 bits, the funct field**."

Walkthrough of R-Type Datapath



4-Stage Pipeline:

1. Instruction Fetch
2. Instruction Decode & Register Fetch
3. Execute

~~(4) Memory Access~~

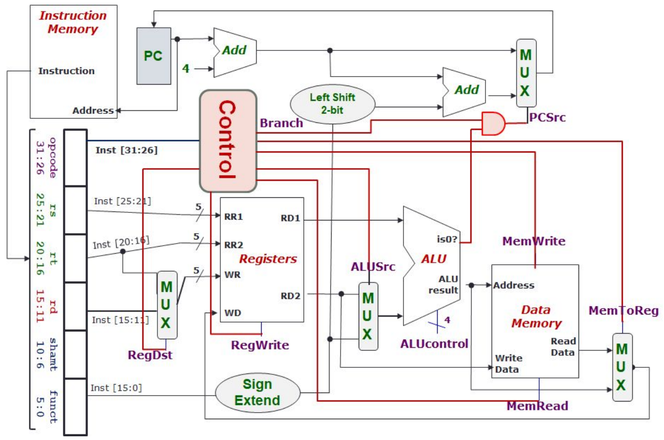
4. Write-Back

Walkthrough of R-Type Datapath

- Let's follow the path of an **instruction** through this diagram.
  - `add $t0, $t1, $t2`
  - $t1 is **register** 9
  - $t2 is **register** 10.
  - Result into Rd ($t0) will be 19.

Stages:

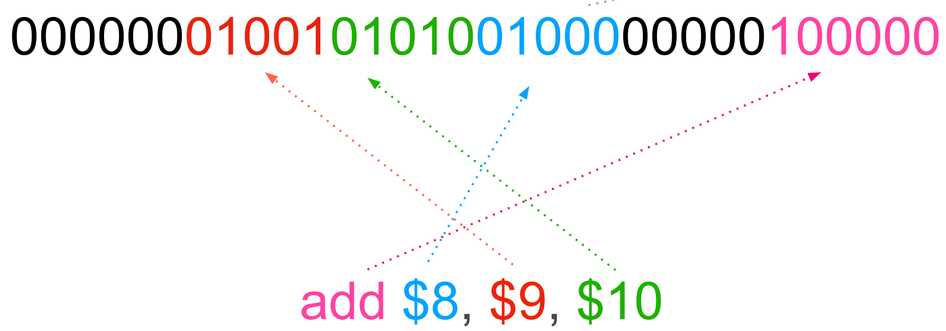
1. Instruction Fetch
2. Instruction Decode & Register Fetch
3. Execute
4. Write-Back

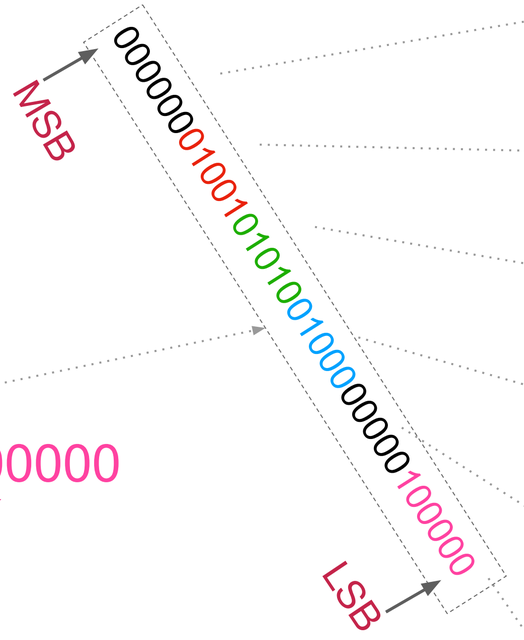


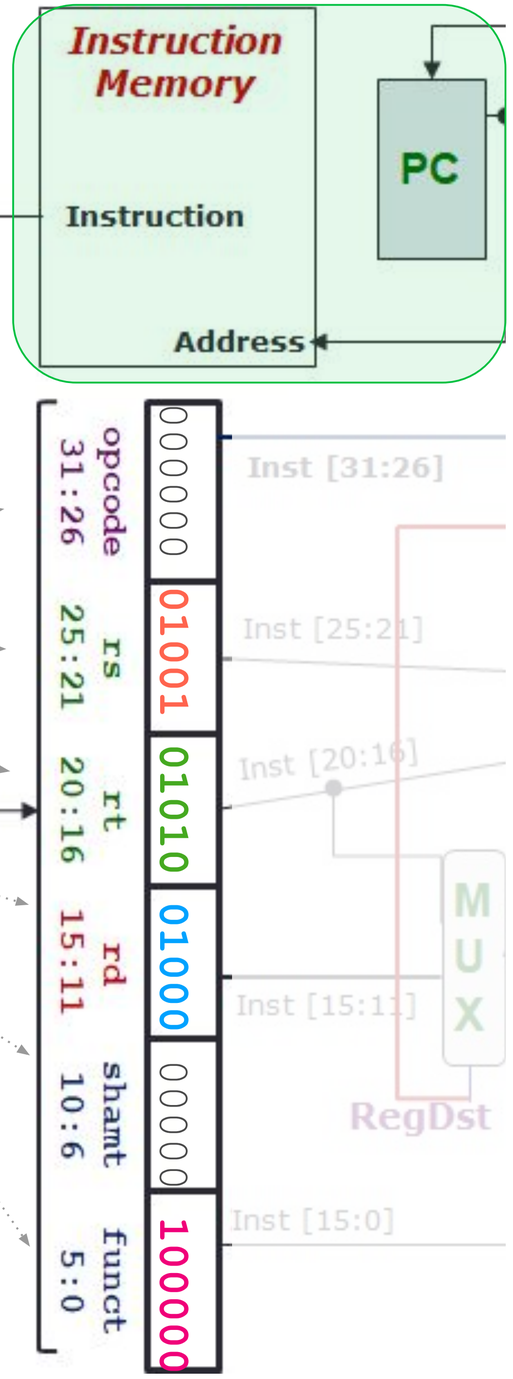
Walkthrough of R-Type Datapath

1: Instruction Fetch:

- The Program Counter (PC) holds the address of the current **instruction**.
- This address is stored in the **Instruction Memory**.
- The **instruction** memory outputs the 32-bit **instruction word**







Walkthrough of R-Type Datapath

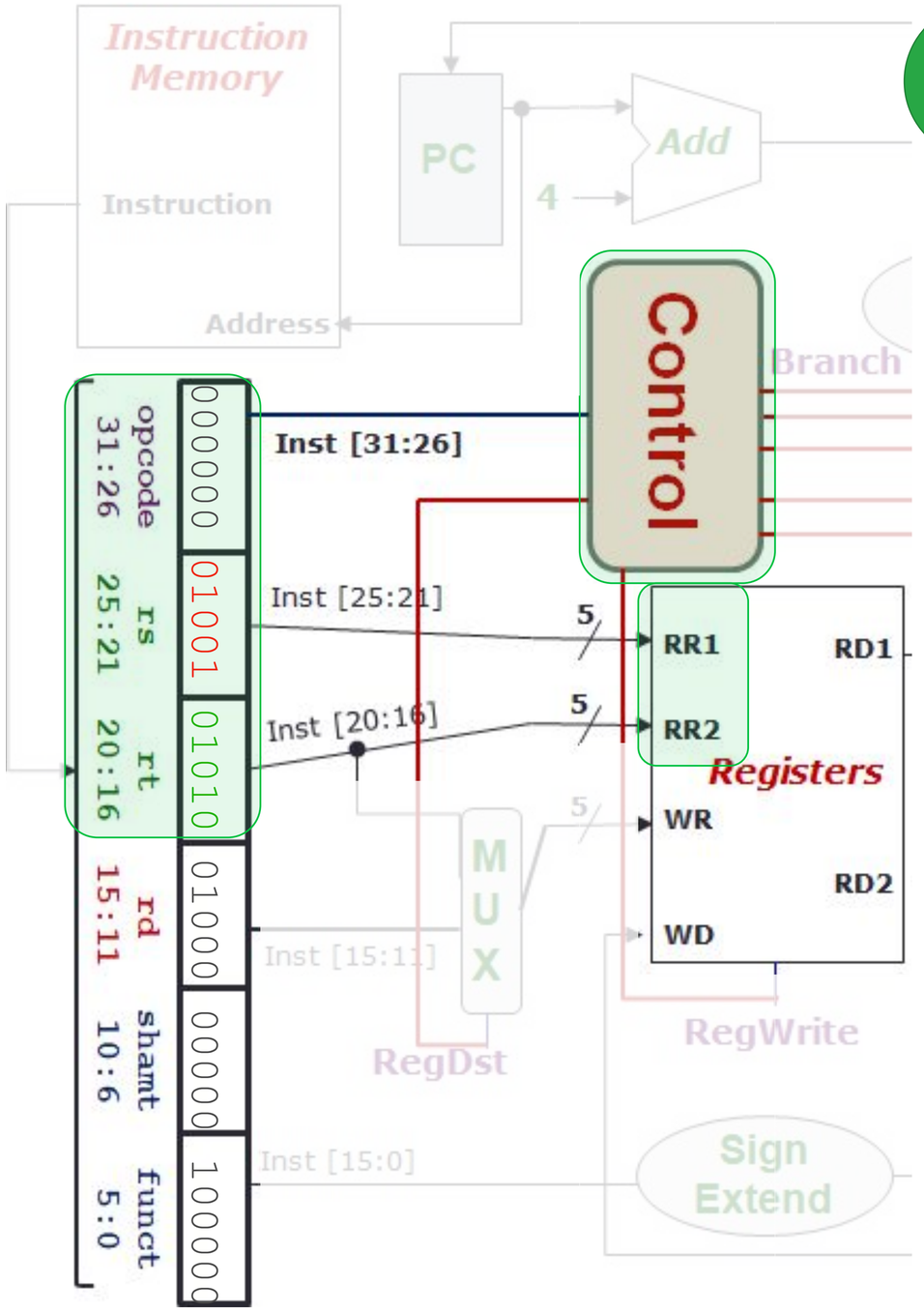
2: Instruction Decode & **Register** Fetch:

The **instruction** bits are split and sent to different units.

opcode [31:26]
(000000) goes to the **Control unit**.

rs [25:21]
(01001 for $t1 or $9)
connects to **Read register 1** (**RR1**) of the **register** file.

rt [20:16]
(01010 for $t2 or $10)
connects to **Read register 2** (**RR2**) of the **register** file.



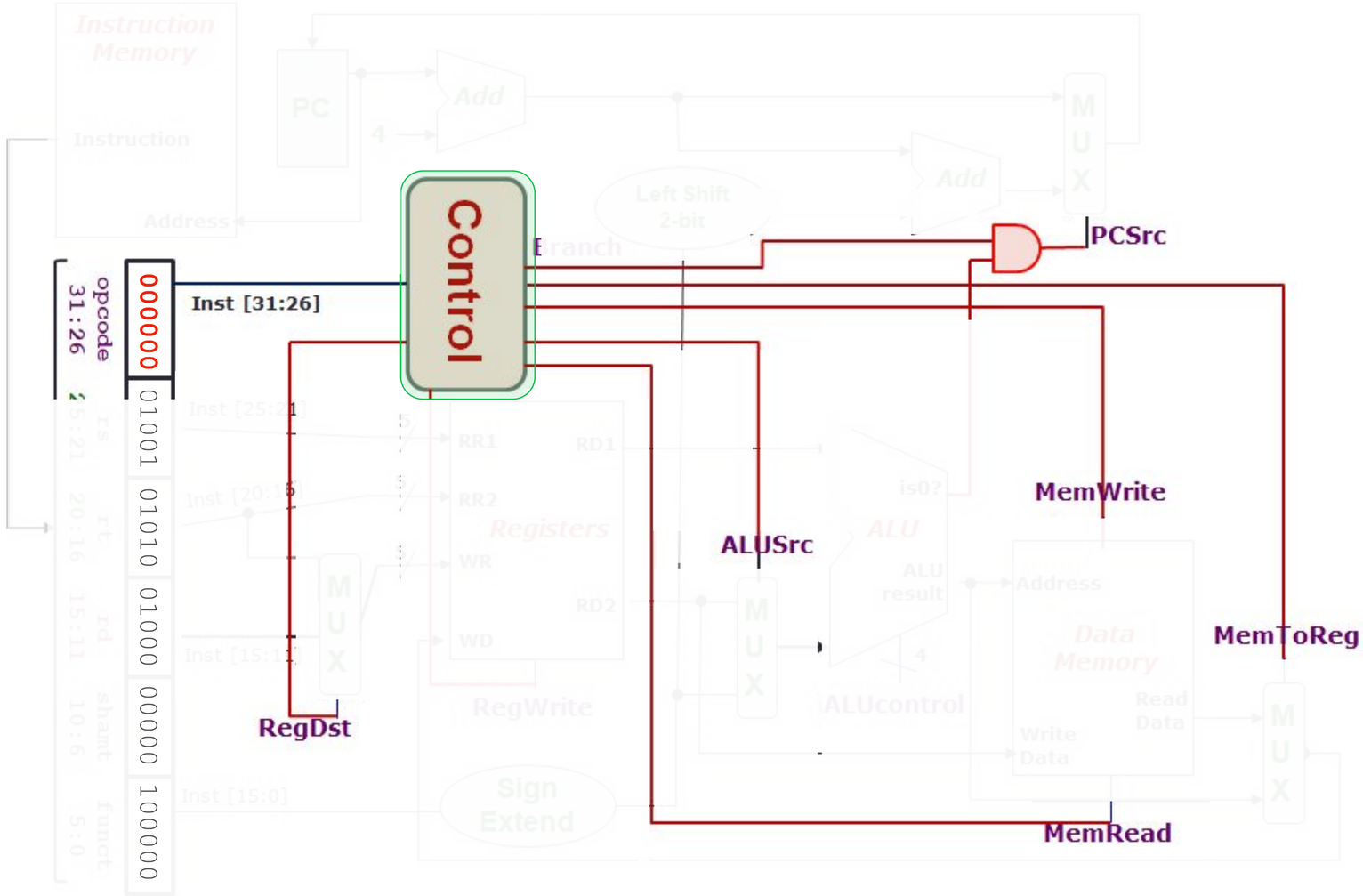
Walkthrough of R-Type Datapath

Instruction Decode & **Register** Fetch: 2

The **Control unit** sees the opcode is **000000** (**R-type**) and sets its output signals accordingly:

RegDst = 1	(Ins-Decode & Reg: 2)
RegWrite = 1	(Ins-Decode & Reg: 2)
ALUSrc = 0	(Ins-Decode & Reg: 2)
MemToReg = 0	(Ins-Decode & Reg: 2)
MemRead = 0	(Ins-Decode & Reg: 2)
MemWrite = 0	(Ins-Decode & Reg: 2)
PCSrc = 0	(Write-back: 4)

If we used a Jump or Immediate Type instruction, the control unit would set different signals.



Walkthrough of R-Type Datapath

Instruction Decode & **Register** Fetch:

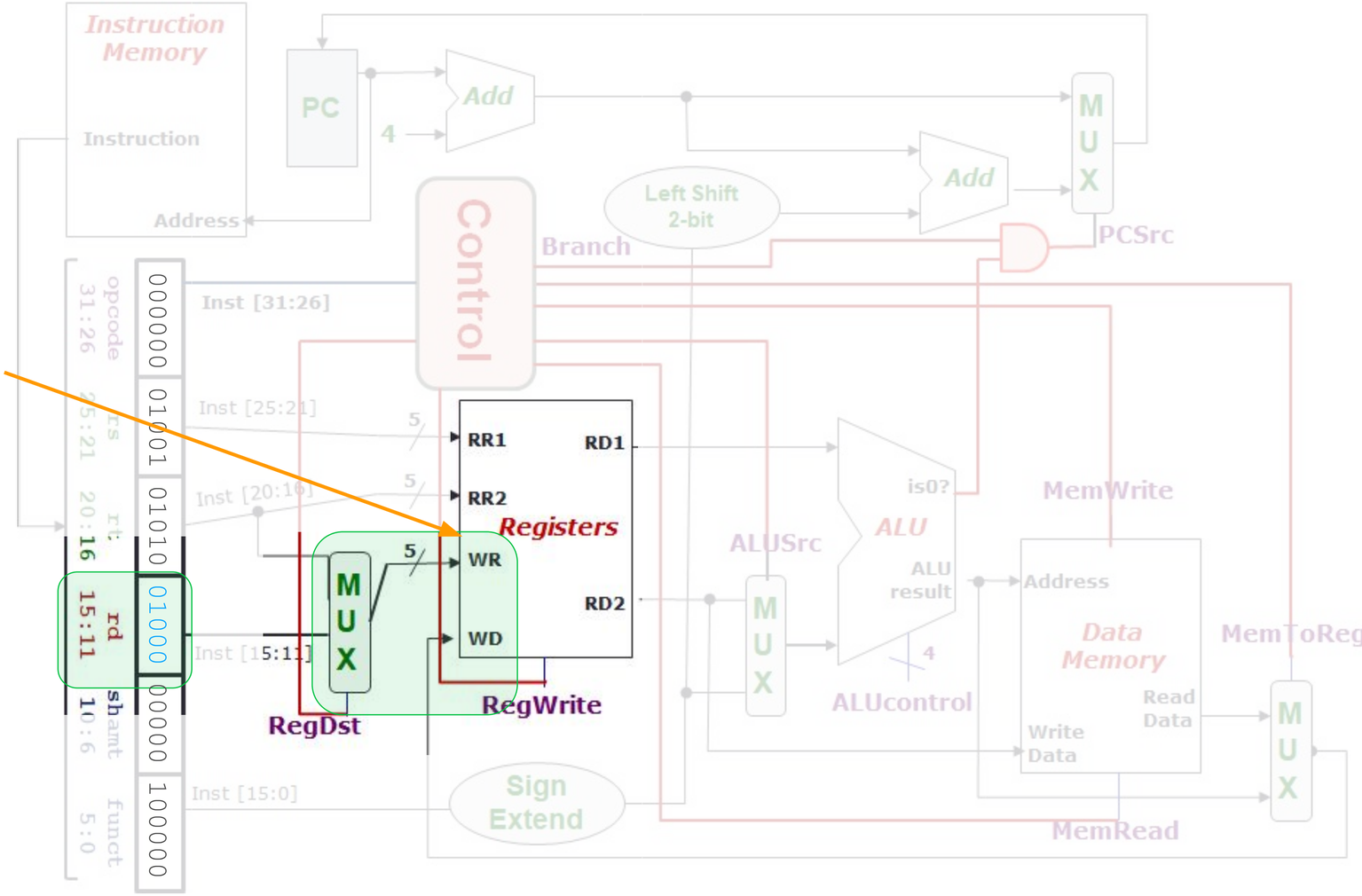
2

The R-Type's **destination register** (Rd):

`rd` [15:11]
(01000 for $t0 or $8)

- Connects to the **Write Register** (**WR**) input.

A key difference from **I-type** instructions.



Walkthrough of R-Type Datapath

Instruction Decode & **Register** Fetch: 2

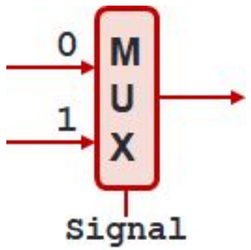
- **RegDst** = 1;
- Controls a MUX,

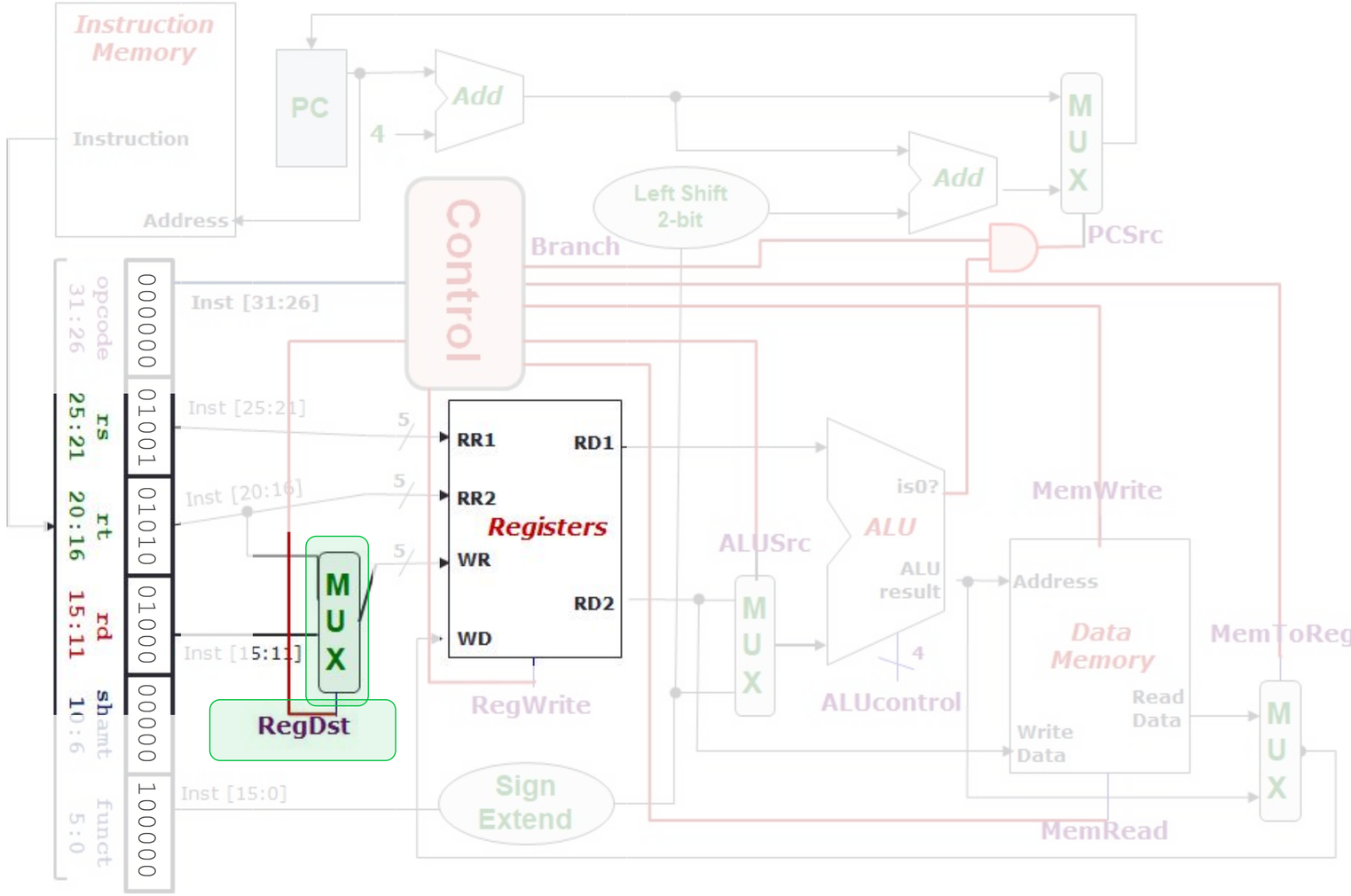
  Instructing to **use the `rd` field**

  instruction [15:11] as **write register**,

  rather than `rt` [20:16].

  - False (0):
    Write **Register** = Inst[20:16] **`rt`**
  - True (1):
    Write **Register** = Inst[15:11] **`rd`**

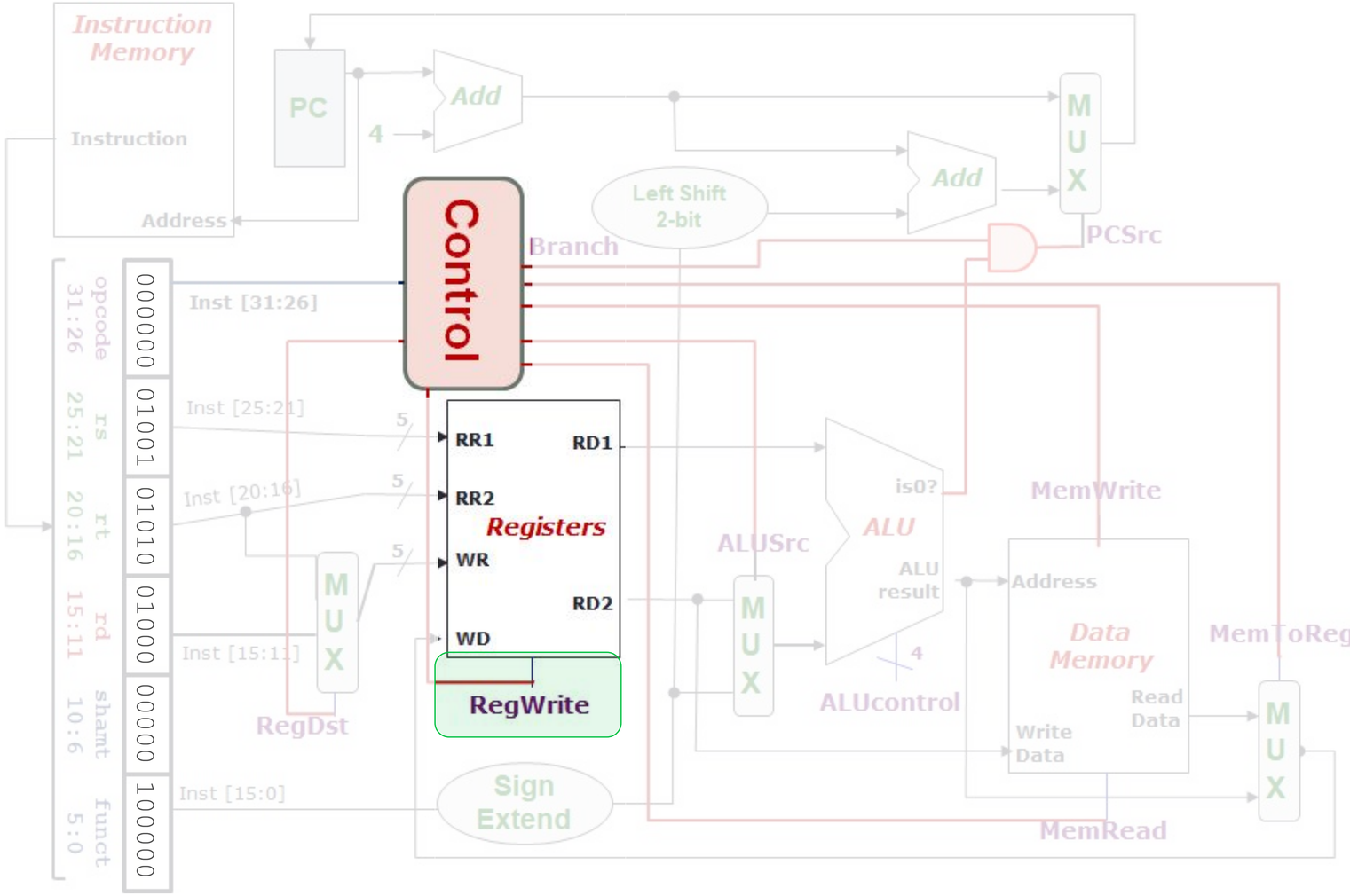




Walkthrough of R-Type Datapath

Instruction Decode & **Register** Fetch: 2

- **RegWrite** = 1,
- Will **write to a register** at the end.
  - False (0): No **register** write
  - True (1): New value will be written



Walkthrough of R-Type Datapath

Instruction Decode & **Register** Fetch: 2

- **ALUSrc** = 0

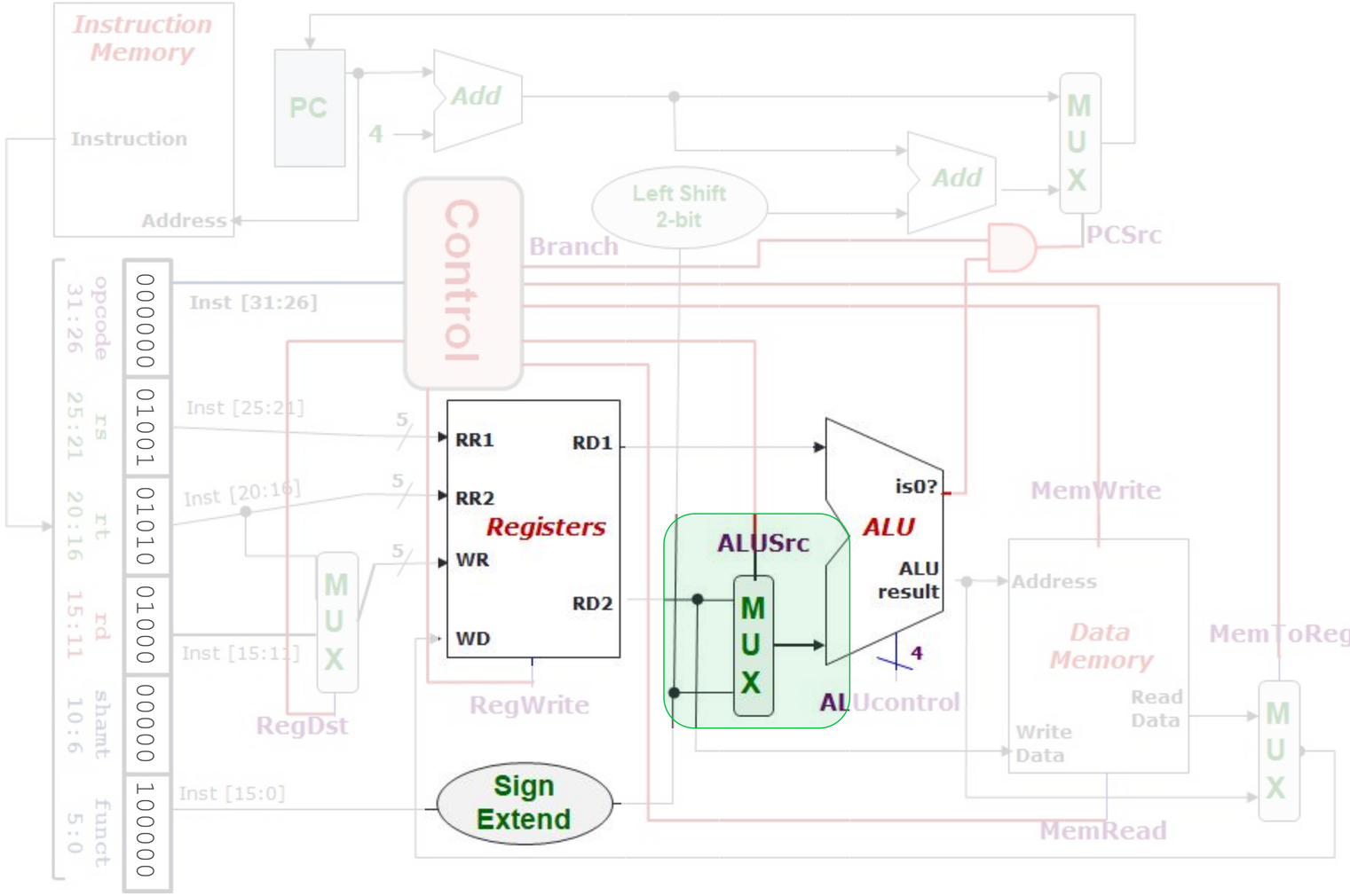
- Controls a MUX,

  to **use the second register read from the register file** as the second **ALU** input,

  rather than the sign-extended immediate value.

False (0): *(R-Type)*
ALUop2 = Read Data 2

True (1): *(I-Type)*
ALUop2 = sign_extend(Inst[15:0])



Walkthrough of R-Type Datapath

Instruction Decode & **Register** Fetch: 2

- **MemToReg** = 0

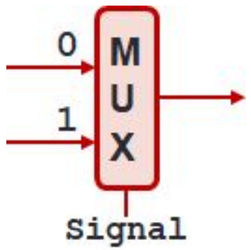
- Controls the final MUX, to write the **ALU result** back to the **register** file, not the data from memory.

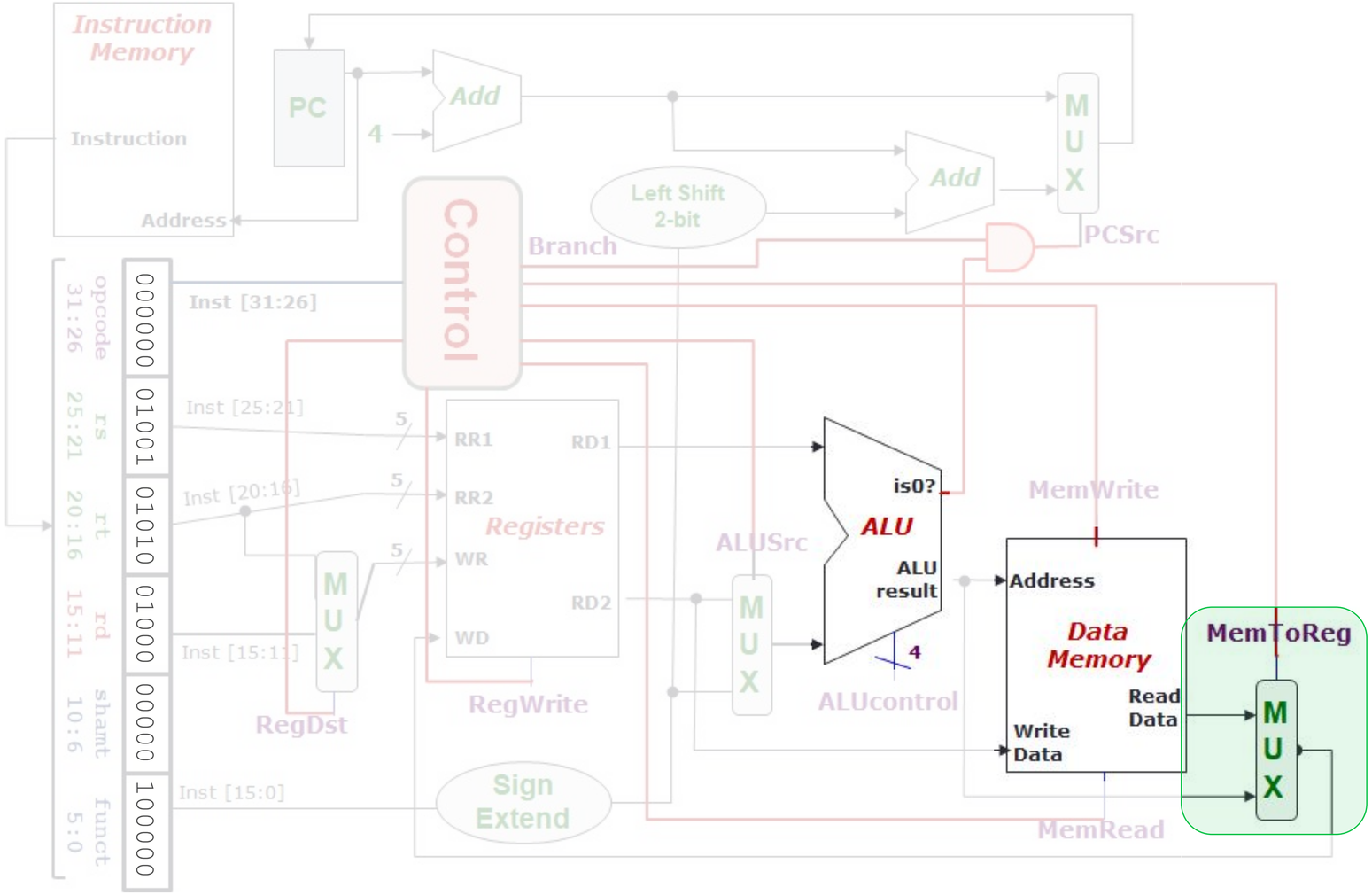
True (1):

Register Write Data = Memory Read Data.

False (0):

Register Write Data = **ALU result**.





Walkthrough of R-Type Datapath

Instruction Decode & **Register** Fetch: 2

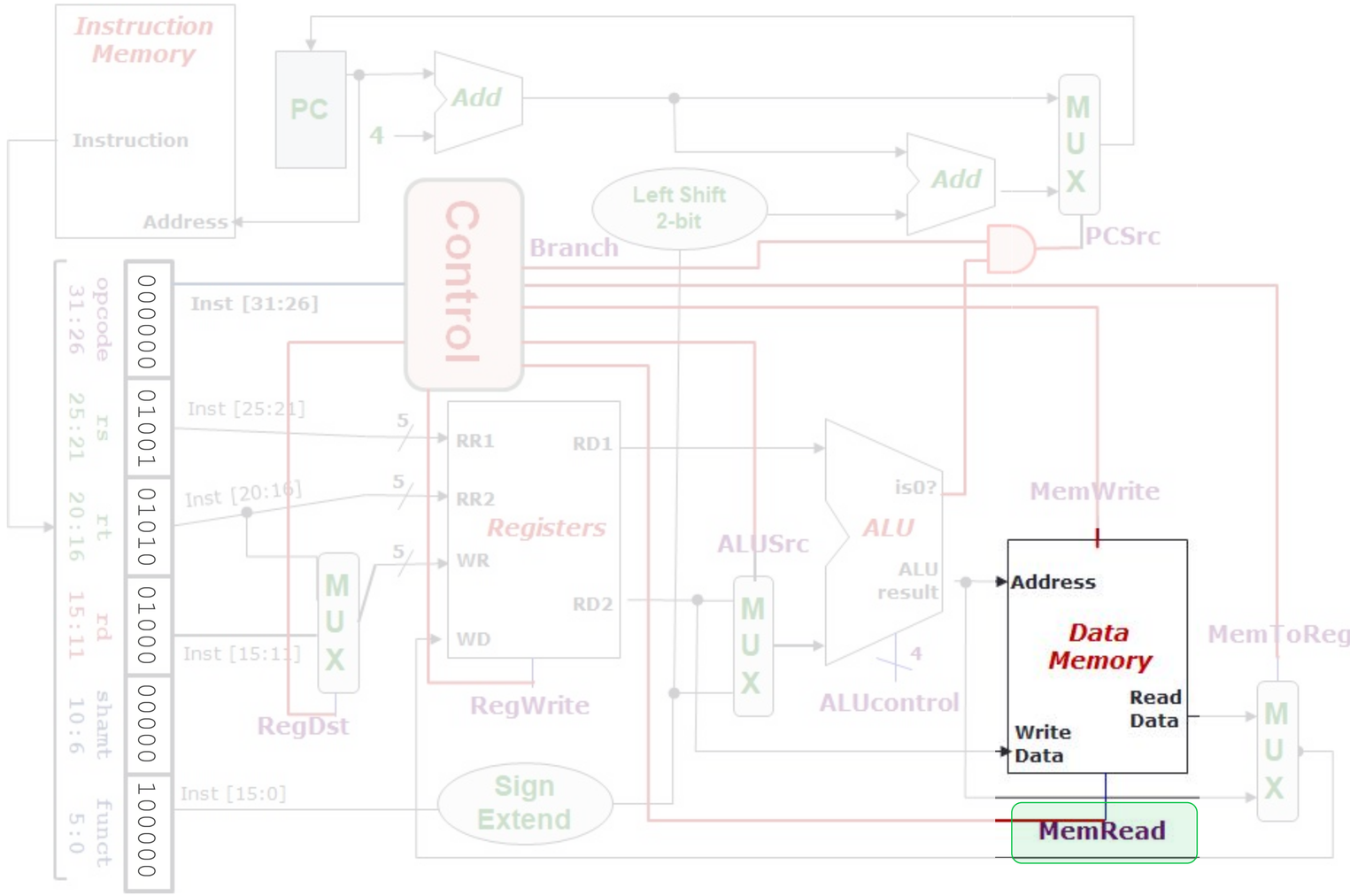
- **MemRead** = 0

False (0):

No memory read access.

True (1):

Read memory using **ALU** result as **Address**.



Walkthrough of R-Type Datapath

Instruction Decode & **Register** Fetch: 2

- **MemWrite** = 0

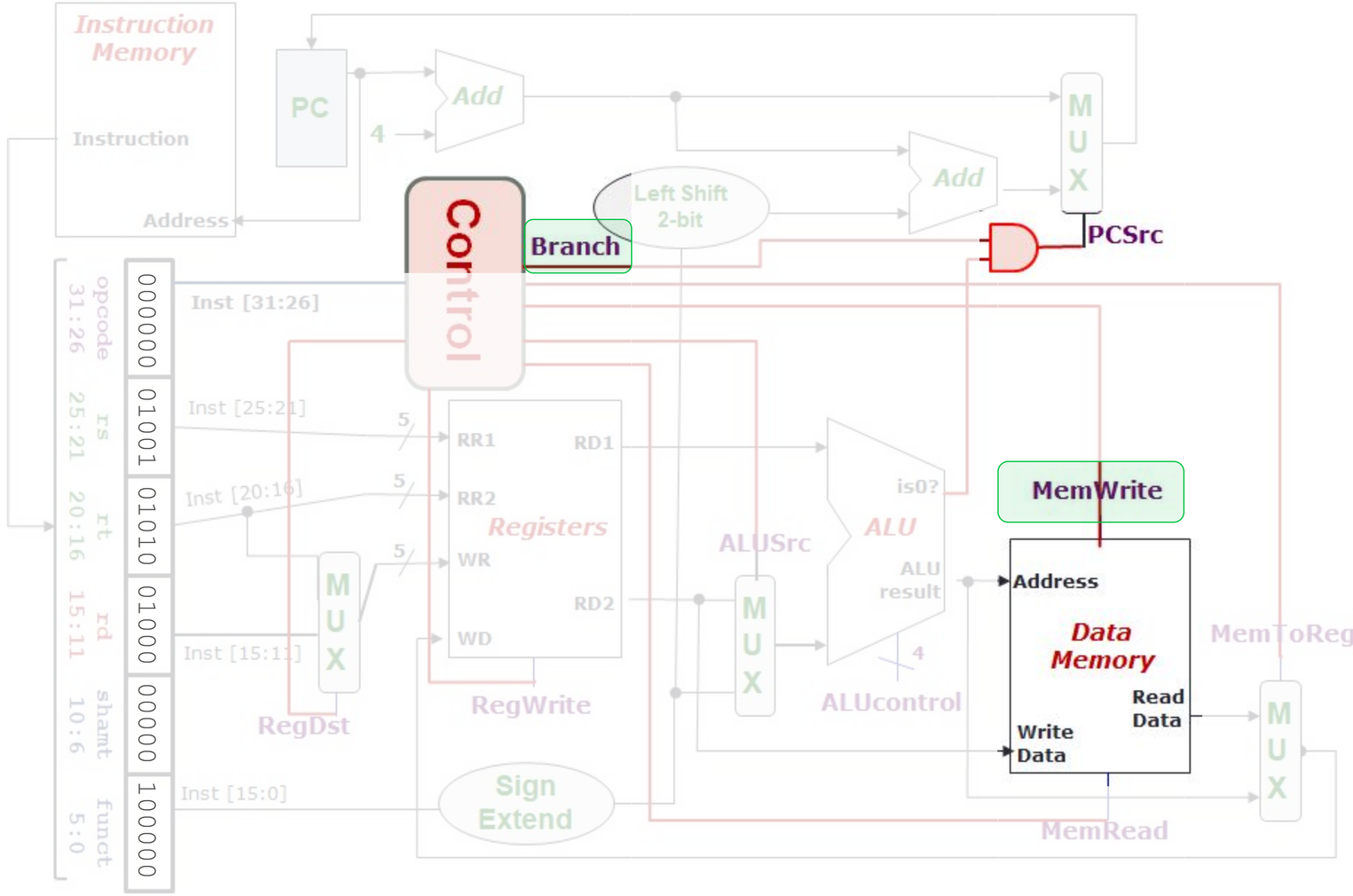
False (0):

No memory write operation.

True (1):

Write to memory at Address (from **ALU** result) with data from **Read Data 2**.

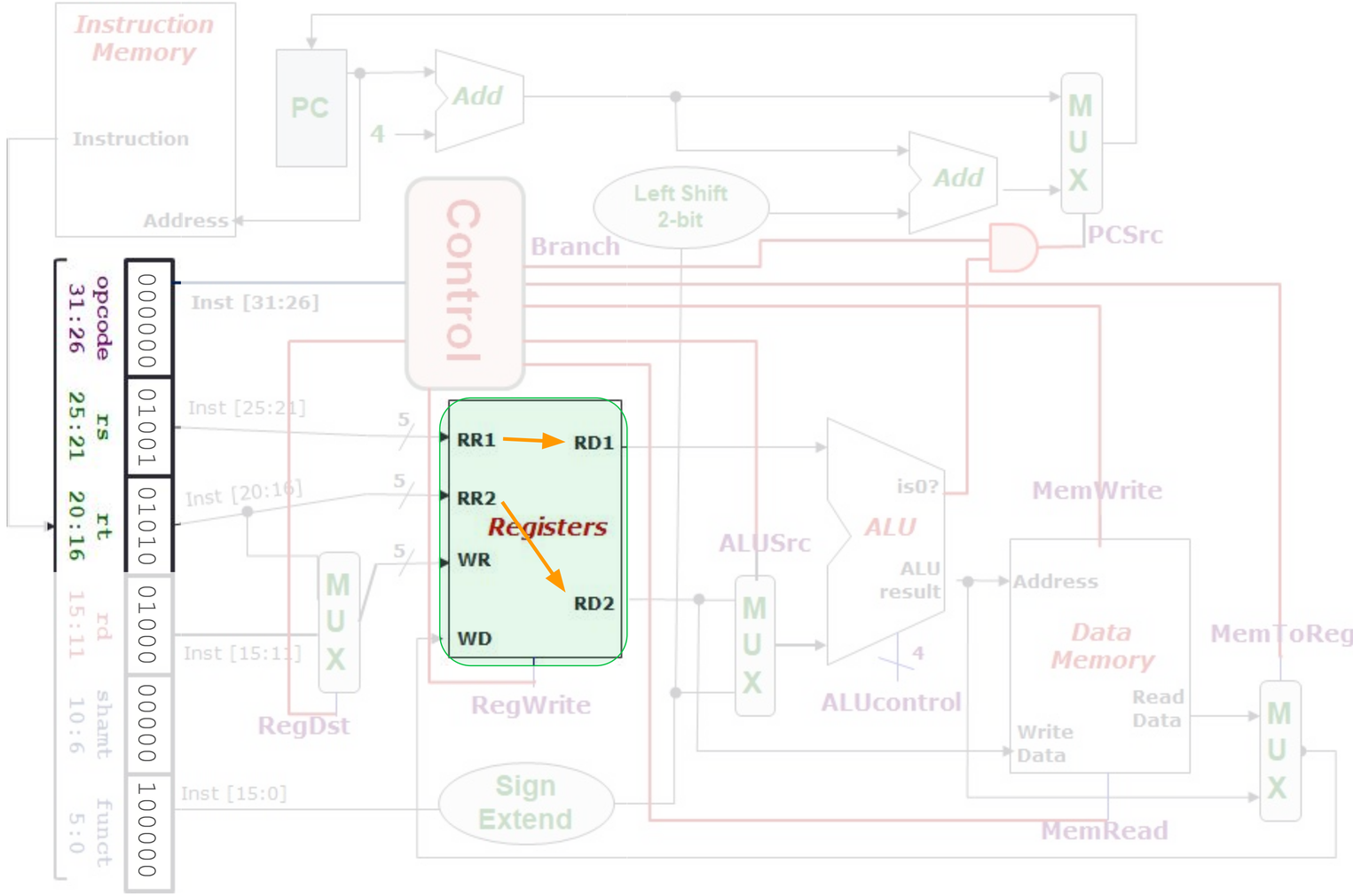
- Branch = 0



Walkthrough of R-Type Datapath

Instruction Decode & **Register** Fetch: 2

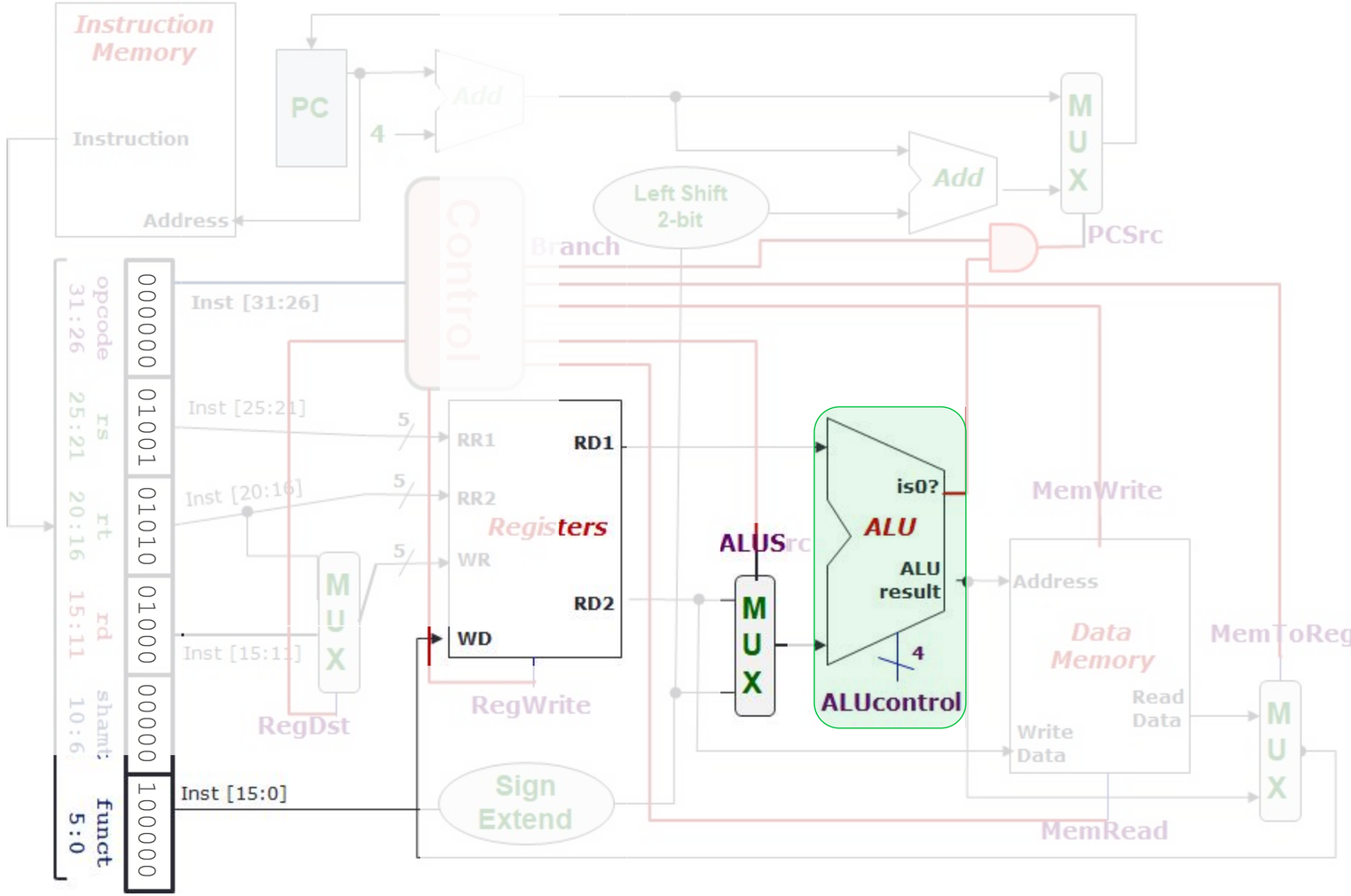
- The **Register File**
  - Reads the values stored in $t1 and in $t2
  - Outputs them on Read Data 1 (**RD1**) and Read Data 2 (**RD2**).



Walkthrough of R-Type Datapath

Execute: 3

- **RD1** (value of **$t1**) and **RD2** (value of **$t2**) are sent to the **ALU**.
- `funct` field [5:0] (**100000** for `add`) is sent to the **ALU Control** unit (MUX).
- **ALU** performs **addition** operation (ALUop = `add`).
- On the two values from the **Register File**.
- Outputs the **ALU result**.



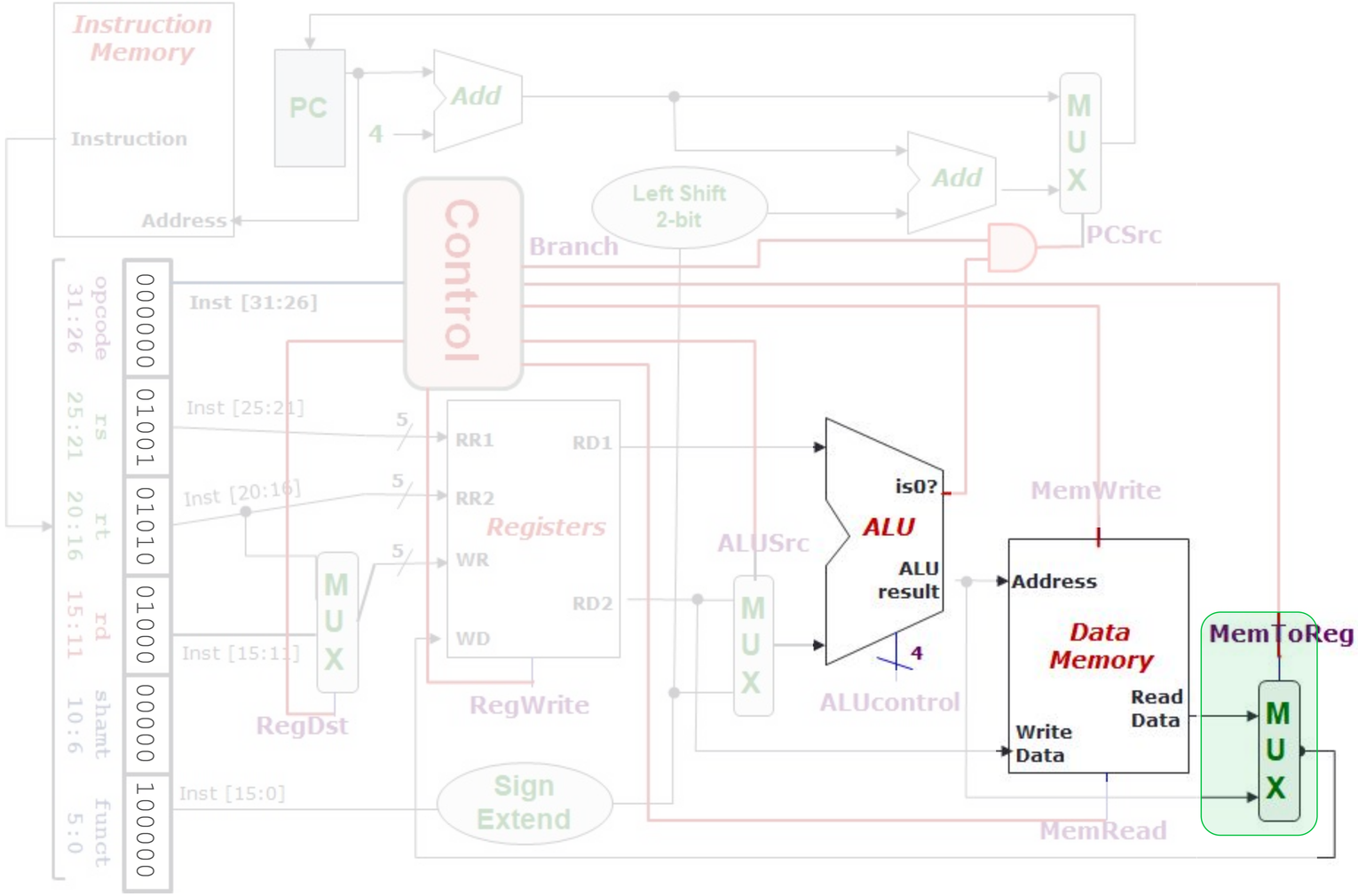
Walkthrough of R-Type Datapath

Write-Back: 4

- The **ALU result**
  (i.e. the sum of $t1 and $t2)

  is fed into the final multiplexer (MUX labeled **MemToReg**).

- The **Control unit** was set to **MemToReg**=0, so the MUX selects the **ALU result** and **(.... next)**



Walkthrough of R-Type Datapath

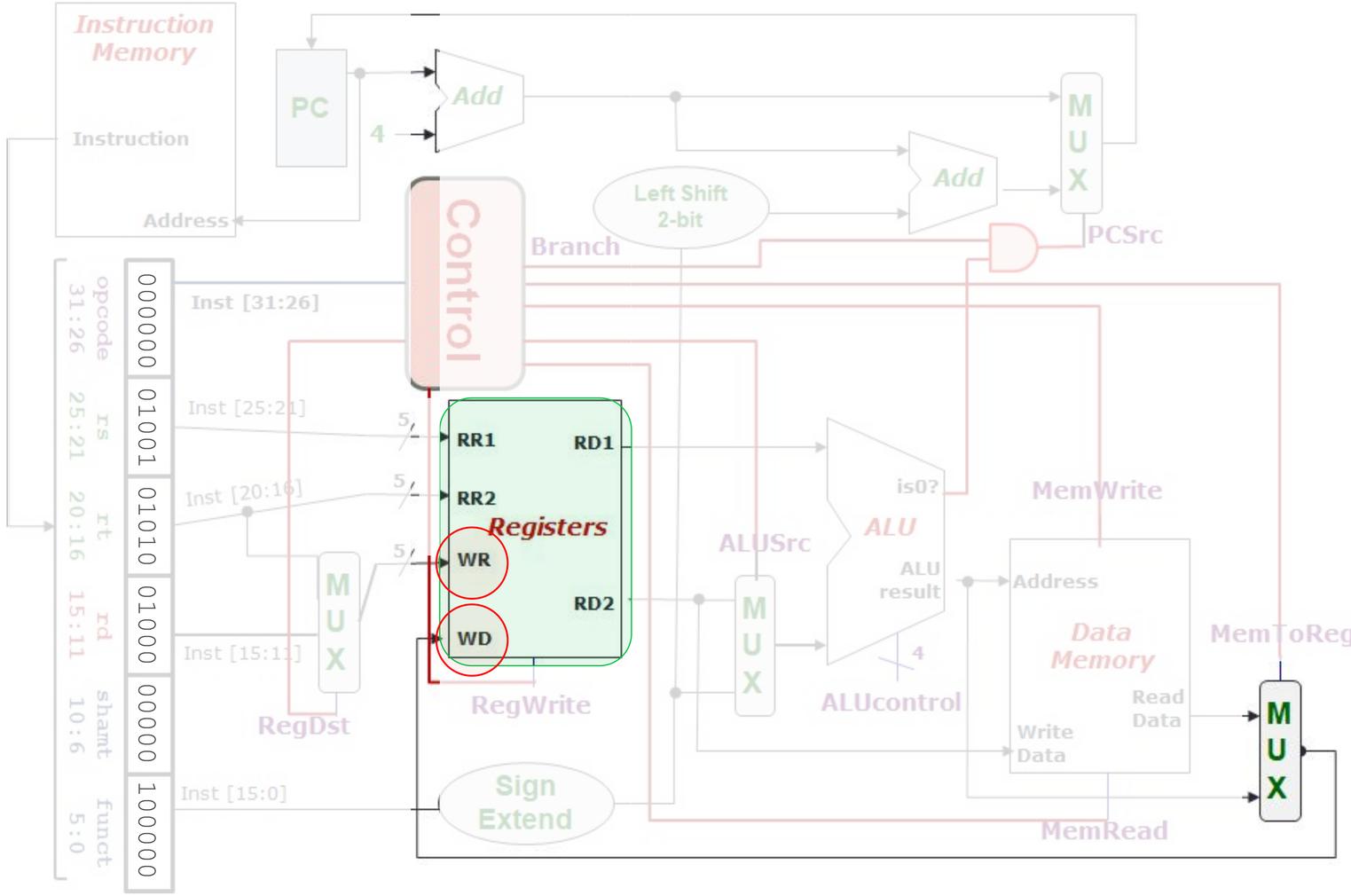
Write-Back: 4

- The selected value is fed back as **Write Data (WD)** into the **Register File (Rd)**.

On the next clock cycle, with RegWrite=1, the value on **WD is written into the register**

Specified by the **Write Register (WR) input**, which is **$t0** (*register position 8*).

```
add $t0, $t1, $t2
```



Walkthrough of R-Type Datapath

Write-Back: 4

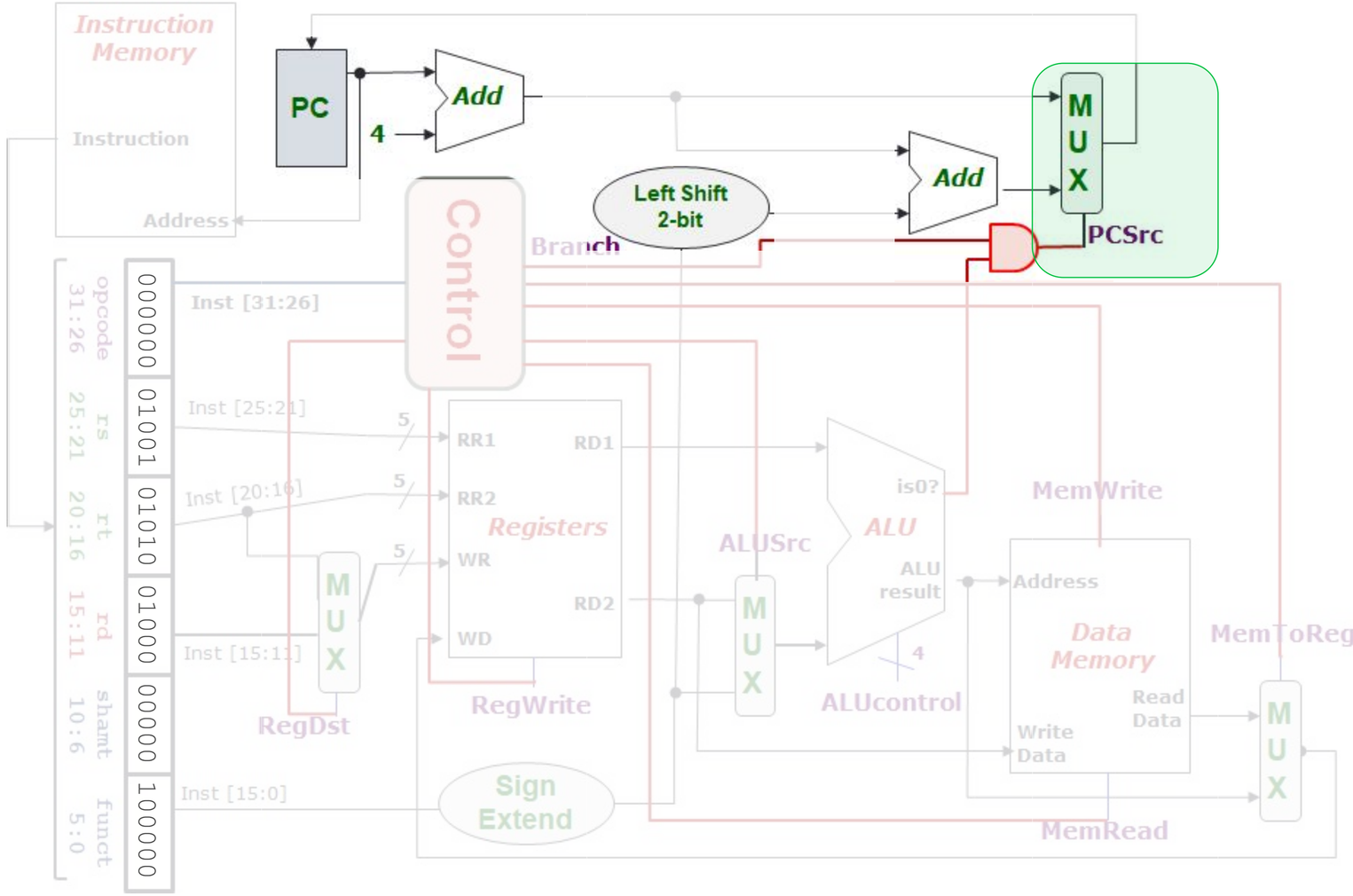
- **PCSrc** = 0

False (0): *(Next instruction +4)*

$PC' = $PC+4.

True (1): *(Jump to i)*

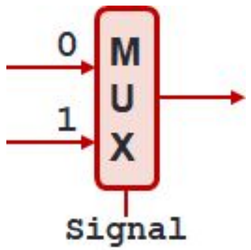
$PC' = ($PC+4)
+(immediate×4).

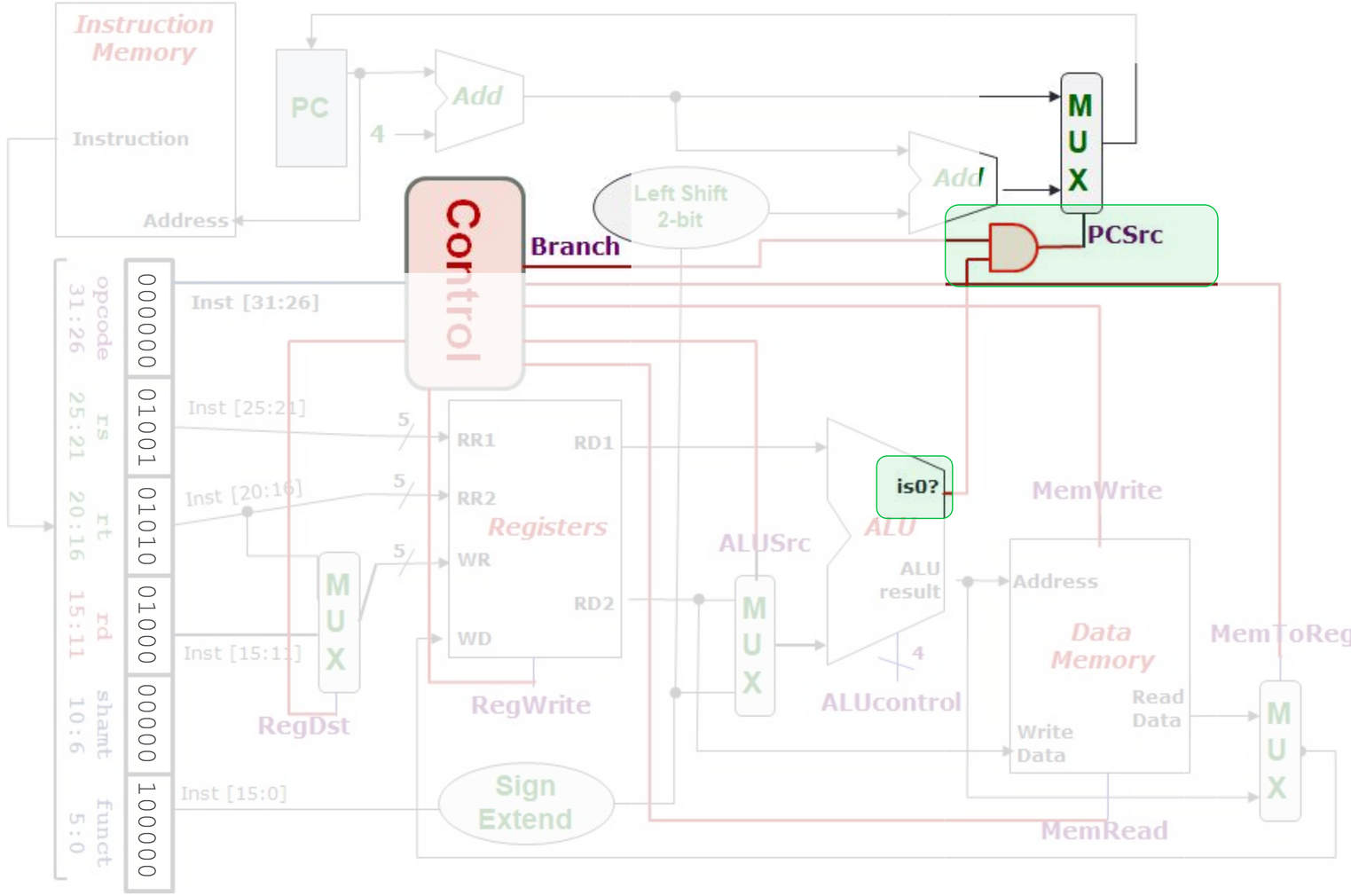


Walkthrough of R-Type Datapath

Write-Back: 4

- Branch
  - False (0):
    Operation is not a branch
  - True (1):
    Operation is a branch
    (e.g., beq).
  - PCSrc = **Branch** & **isZero**?





Overview of R-Type Datapath

add $t0, $t1, $t2,

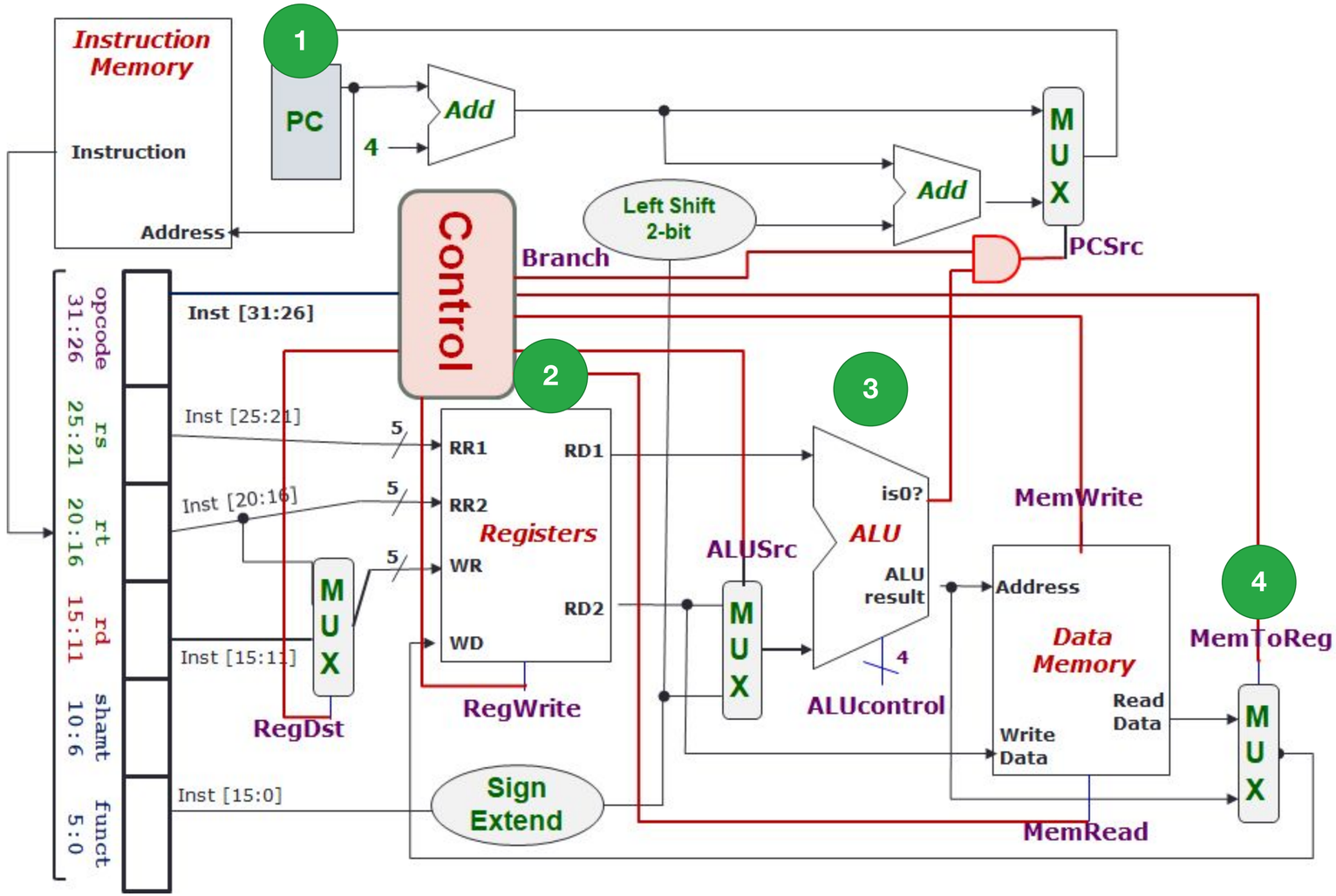
One instruction through the datapath:

1. **Fetch:**
   Instruction word leaves Instruction Memory.

2. **Decode:**
   Control reads opcode 000000; $t1 and $t2 are read.

3. **Execute:**
   ALU adds the two register values.

4. **Write-Back:**
   Result is written into $t0.



I-Type Instructions

with

Load Upper Intermediate (`lui`) and

Or Intermediate (`ori`)

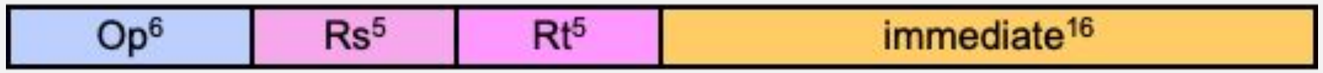
(Example that builds into a Pseudo-Instruction)

I-Type Instruction Format

- **I-Type** (Immediate-Type)
  - Purpose: Use a constant value (immediate) in the **instruction** itself, e.g.,

    `addi $t1, $t2, 25`.

  - Format:



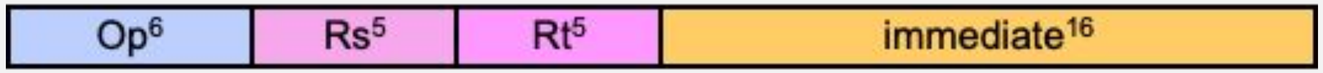
  - Why **Op** is NOT 0:
    - For **I-type** instructions, the **Op** field uniquely identifies the **instruction**.
    - There is no **funct** field.
    - The CPU sees the **Op** code and knows exactly what to do.

- Example: **Op** code 001000 means `addi`, 001101 means `ori`.
  - **Rs**: Source **register**
  - **Rt**: Destination **register** (notice this is different from R-type's **Rd**)
  - **immediate**: A 16-bit constant value

I-Type Instruction Format

- Constants are used quite frequently in programs.
  - The R-type shift instructions have a 5-bit shift amount constant.
  - What about other instructions that need a constant?

- **I-Type**: Instructions with **immediate** Operands.



- A 16-bit immediate constant is stored inside the **instruction**
  - **Rs** is the source **register** number
  - **Rt** is now the destination **register** number (for R-type, it was **Rd**)

- Examples of **I-Type ALU** Instructions:

Add immediate:	**addi $t1, $t2, 5**	**# $t1 = $t2 + 5**
OR immediate:	**ori $t1, $t2, 5**	**# $t1 = $t2 | 5**

I-Type ALU Instructions

Instruction	Meaning	Op	Rs	Rt	Immediate
`addi $t1, $t2, 25`	`$t1 = $t2 + 25`	0x8	$t2	$t1	25
`addiu $t1, $t2, 25`	`$t1 = $t2 + 25`	0x9	$t2	$t1	25
`andi $t1, $t2, 25`	`$t1 = $t2 & 25`	0xc	$t2	$t1	25
`ori $t1, $t2, 25`	`$t1 = $t2 | 25`	0xd	$t2	$t1	25
`xori $t1, $t2, 25`	`$t1 = $t2 ^ 25`	0xe	$t2	$t1	25
`lui $t1, 25`	`$t1 = 25 << 16`	0xf	0	$t1	25

- **`addi`**: **overflow causes an** arithmetic exception
  - In **case of overflow**, the result is not written to the destination **register**
- **`addiu`**: same operation as **`addi`**, but **overflow is ignored**
- Immediate constant for **`addi`** and **`addiu`** is signed
  - No need for **`subi`** or **`subiu`** instructions
- Immediate constant for **`andi`**, **`ori`**, **`xori`** is unsigned

MIPS doesn't have a **`subi`** **instruction**!

I-Type ALU Instructions

Instruction	Meaning	Op	Rs	Rt	Immediate
`addi  $t1, $t2, 25`	`$t1 = $t2 + 25`	0x8	$t2	$t1	25
`addiu $t1, $t2, 25`	`$t1 = $t2 + 25`	0x9	$t2	$t1	25
`andi $t1, $t2, 25`	`$t1 = $t2 & 25`	0xc	$t2	$t1	25
`ori  $t1, $t2, 25`	`$t1 = $t2 | 25`	0xd	$t2	$t1	25
`xori $t1, $t2, 25`	`$t1 = $t2 ^ 25`	0xe	$t2	$t1	25
`lui  $t1, 25`	`$t1 = 25 << 16`	0xf	0	$t1	25

- `lui $t1, 0x1234`
  - Loads the 16-bit immediate value into the upper half (the most significant 16 bits**) of the destination register** `$t1` and sets the lower 16 bits to zero.
- Result: **$t1** becomes **0x12340000**.
- Since the immediate field in **I-type** instructions is only 16 bits, we can't load a full 32-bit constant (like a memory address**) in one instruction.**

I-Type ALU Instructions

Instruction	Meaning	Op	Rs	Rt	Immediate
`addi  $t1, $t2, 25`	`$t1 = $t2 + 25`	0x8	$t2	$t1	25
`addiu $t1, $t2, 25`	`$t1 = $t2 + 25`	0x9	$t2	$t1	25
`andi $t1, $t2, 25`	`$t1 = $t2 & 25`	0xc	$t2	$t1	25
`ori  $t1, $t2, 25`	`$t1 = $t2 | 25`	0xd	$t2	$t1	25
`xori $t1, $t2, 25`	`$t1 = $t2 ^ 25`	0xe	$t2	$t1	25
`lui  $t1, 25`	`$t1 = 25 << 16`	0xf	0	$t1	25

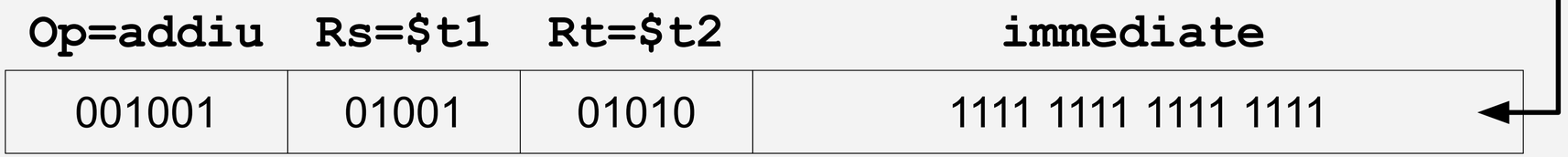
`li $t1 0x10011234` → Pseudo instruction becomes the following two real MIPS instructions.

- `lui` is the first step of a two-step process:
  - `lui $t1, 0x1001` `# Load the upper part of the address: $t1 = 0x10010000`
  - `ori $t1, $t1, 0x1234` `# OR in the lower part: $t1 = 0x10010000 | 0x00001234 = 0x10011234`

- Now $t1 holds the full **32-bit address 0x10011234**.

Examples of I-Type ALU Instructions

Expression	Equivalent MIPS Instruction
`A = B + 5;`	`addiu $t0, $t1, 5`
`C = B - 1;`	`addiu $t2, $t1, -1`
`A = B & 0xf;`	`andi  $t0, $t1, 0xf`
`C = B | 0xf;`	`ori  $t2, $t1, 0xf`
`C = 5;`	`addiu $t2, $zero, 5`
`A = B;`	`addiu $t0, $t1, 0`



- No need for **subiu**, because **addiu** has signed **immediate**
- **Register** $zero has always the value 0

Examples of I-Type ALU Instructions

MIPS Instruction Types

Type	-31- format (bits) -0-					
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	opcode (6)	rs (5)	rt (5)	immediate (16)		
J	opcode (6)	address (26)				

R-Type Instructions (Opcode 000000)

Main processor instructions that do not require a target address, immediate value, or branch displacement use an R-type coding format. This format has fields for specifying of up to three registers and a shift amount.

For instructions that do not use all of these fields, the unused fields are coded with all 0 bits. All R-type instructions use a 000000 opcode. The operation is specified by the function field.

opcode (6)	rs (5)	rt (5)	rd (5)	sa (5)	function (6)

Instruction		Function
add	rd, rs, rt	100000
addu	rd, rs, rt	100001
and	rd, rs, rt	100100
break		001101
div	rs, rt	011010
divu	rs, rt	011011
jalr	rd, rs	001001
jr	rs	001000
mfhi	rd	010000
mflo	rd	010010
mthi	rs	010001
mtlo	rs	010011
mult	rs, rt	011000
multu	rs, rt	011001
nor	rd, rs, rt	100111
or	rd, rs, rt	100101
sll	rd, rt, sa	000000
sllv	rd, rt, rs	000100
slt	rd, rs, rt	101010
sltu	rd, rs, rt	101011
sra	rd, rt, sa	000011
srav	rd, rt, rs	000111
srl	rd, rt, sa	000010
srlv	rd, rt, rs	000110
sub	rd, rs, rt	100010
subu	rd, rs, rt	100011
syscall		001100
xor	rd, rs, rt	100110

Expression	Equivalent MIPS Instruction
`A = B + 5;`	`addiu $t0, $t1, 5`
`C = B - 1;`	`addiu $t2, $t1, -1`
`A = B & 0xf;`	`andi $t0, $t1, 0xf`
`C = B | 0xf;`	`ori $t2, $t1, 0xf`
`C = 5;`	`addiu $t2, $zero, 5`
`A = B;`	`addiu $t0, $t1, 0`

Op=addiu	Rs=$t1	Rt=$t2	immediate
001001	01001	01010	1111 1111 1111 1111





I-Type Instructions (All opcodes except 000000, 00001x, and 0100xx)

I-type instructions have a 16-bit immediate field that codes an immediate operand, a branch target offset, or a displacement for a memory operand. For a branch target offset, the immediate field contains the signed difference between the address of the following instruction and the target label, with the two low order bits dropped. The dropped bits are always 0 since instructions are word-aligned.

For the `bgez`, `bgtz`, `blez`, and `bltz` instructions, the rt field is used as an extension of the opcode field.

opcode (6)	rs (5)	rt (5)	immediate (16)

Instruction		Opcode	Notes
addi	rt, rs, immediate	001000	
addiu	rt, rs, immediate	001001	
andi	rt, rs, immediate	001100	
beq	rs, rt, label	000100	
bgez	rs, label	000001	rt = 00001
bgtz	rs, label	000111	rt = 00000
blez	rs, label	000110	rt = 00000
bltz	rs, label	000001	rt = 00000
bne	rs, rt, label	000101	
lb	rt, immediate(rs)	100000	
lbu	rt, immediate(rs)	100100	
lh	rt, immediate(rs)	100001	
lhu	rt, immediate(rs)	100101	
lui	rt, immediate	001111	
lw	rt, immediate(rs)	100011	
lwc1	rt, immediate(rs)	110001	
ori	rt, rs, immediate	001101	
sb	rt, immediate(rs)	101000	
slti	rt, rs, immediate	001010	
sltiu	rt, rs, immediate	001011	
sh	rt, immediate(rs)	101001	
sw	rt, immediate(rs)	101011	
swc1	rt, immediate(rs)	111001	
xori	rt, rs, immediate	001110	

I-Type ALU Instructions

Exercise

- Given that: MIPS architecture, `$t1 = 5`

addi $t0, $t1, 0xffff # $t0 = 0x_____

Op	rs	rt	immediate		
_____	_____	_____	_____	_____	_____

addiu $t2, $t1, -1024 # $t2 = 0x_____

Op	rs	rt	immediate		
_____	_____	_____	_____	_____	_____



Exercise

I-Type ALU Instructions

- Given that: MIPS architecture, `$t1` = 5

ori $t2, $t1, 0xf # $t2 = 0x_____

Op	rs	rt	immediate		
_____	_____	_____	_____	_____	_____

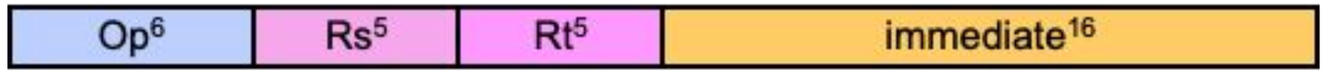
andi $t0, $t1, 0xf # $t0 = 0x_____

Op	rs	rt	immediate		
_____	_____	_____	_____	_____	_____

Instruction Type	Immediate Extension
addi, addiu, slti, sltiu, lw, sw, lb, sb, etc.	Sign-extend
andi, ori, xori	Zero-extend



32-bit Constants



- **I-Type** instructions can have only 16-bit constants

- What if we want to load a 32-bit constant into a **register**?
- Can't have a 32-bit constant in **I-Type** instructions ☹
  - The sizes of all instructions are fixed to 32 bits
- Solution: use two instructions instead of one ☺
- Suppose we want: $t1 = 0xabcd1234 (32-bit constant)

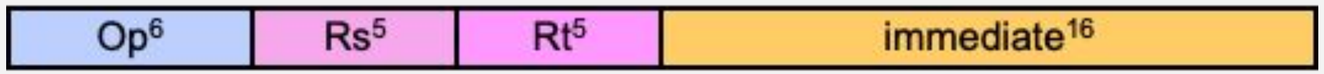
lui: load upper immediate

lui $t1, 0xabcd

ori $t1, $t1, 0x1234

	Upper 16 bits	Lower 16 bits
# $t1	0xabcd	0x0000
# $t1	0xabcd	0x1234

32-bit Constants



- The Solution: Two Instructions (**lui** + **ori**)

- To load a 32-bit number, we use two separate instructions:
  - One to set the upper 16 bits of the target **register**.
  - One to set the lower 16 bits of the target **register**.

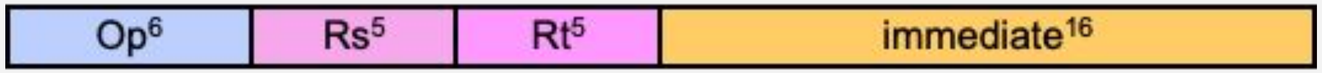
lui: load upper immediate

lui $t1, 0xabcd

ori $t1, $t1, 0x1234

	Upper 16 bits	Lower 16 bits
# $t1	0xabcd	0x0000
# $t1	0xabcd	0x1234

32-bit Constants



- The Solution: Two Instructions (**lui** + **ori**)

- The **lui** (Load Upper Immediate) **Instruction**
- Purpose: **lui** $rt, immediate loads a 16-bit immediate value into the upper 16 bits of the target **register** ($rt).
  - It then sets the lower 16 bits to zero.
  - The upper half (abcd) is in place, and the lower half is zeros (0000).

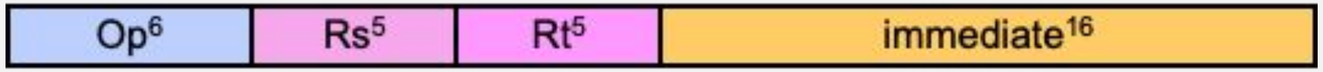
lui： load upper immediate

lui $t1, 0xabcd

ori $t1, $t1, 0x1234

	Upper 16 bits	Lower 16 bits
# $t1	0xabcd	0x0000
# $t1	0xabcd	0x1234

32-bit Constants



- The Solution: Two Instructions (**lui** + **ori**)

- Completing the Load with **ori** (Or Immediate)
- The **lui instruction** zeroed out the lower 16 bits.
  - We now need to put our desired lower 16 bits (`0x1234`) into the **register** without disturbing the upper bits we just set.

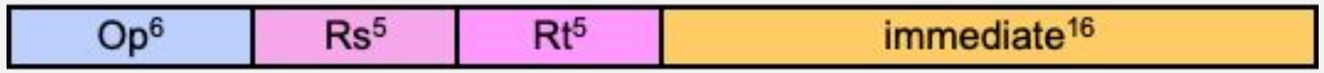
lui: load upper immediate

lui $t1, 0xabcd

ori $t1, $t1, 0x1234

	Upper 16 bits	Lower 16 bits
# $t1	0xabcd	0x0000
# $t1	0xabcd	0x1234

32-bit Constants



- The Solution: Two Instructions (**lui** + **ori**)

- We use the **ori** (Or Immediate) **instruction** for this. The logical OR operation is perfect:
  0xabcd0000 OR 0x00001234 = 0xabcd1234
- The **instruction** would be: ori $t1, $t1, 0x1234

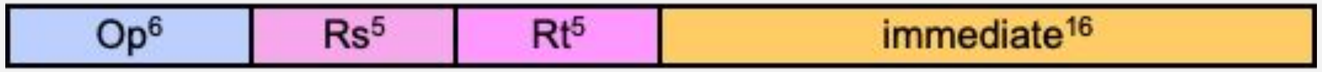
lui: load upper immediate

lui $t1, 0xabcd

ori $t1, $t1, 0x1234

	Upper 16 bits	Lower 16 bits
# $t1	0xabcd	0x0000
# $t1	0xabcd	0x1234

32-bit Constants



- The Solution: Two Instructions (**lui** + **ori**)

- Take current value of $t1 (**0xabcd0000**), ORs it with **0x1234**,
- Stores the result (**0xabcd1234**) back into $t1.

```
lui: load upper immediate

lui $t1, 0xabcd
ori $t1, $t1, 0x1234
```

	Upper 16 bits	Lower 16 bits
# $t1	0xabcd	0x0000
# $t1	0xabcd	0x1234

More Pseudo-Instructions

Pseudo Instructions

- Introduced by the assembler as if they were real instructions.
- Facilitate assembly language programming.

Pseudo-Instruction	Equivalent MIPS Instruction
`move $t1, $t2`	`addu $t1, $t2, $zero`
`not $t1, $t2`	`nor $t1, $t2, $zero`
`neg $t1, $t2`	`sub $t1, $zero, $t2`
`li $t1, -5`	`addiu $t1, $zero, -5`
`li $t1, 0xabcd1234`	`lui $t1, 0xabcd`
	`ori $t1, $t1, 0x1234`

- The MARS tool has a long list of pseudo-instructions.

Pseudo Instructions

What are Pseudo-Instructions?

- They are instructions that do not exist in the actual MIPS hardware instruction set.
- They are provided by the assembler (the program that converts text code into machine code) as a convenience for the programmer.
- The assembler sees a pseudo-**instruction** and automatically replaces it with one or more real, valid MIPS instructions.

Pseudo Instructions

Why are they useful?

- They make code easier to read and write.
- They act as helpful shortcuts, saving the programmer from writing longer, more complex sequences of real instructions.

Pseudo Instructions

Examples from the Table: **`move $t1, $t2`**

- Meaning: Copy the value from **register `$t2`** into **register `$t1`**.
- Implementation: **`addu $t1, $t2, $zero`**
- How it works: It adds the value in **`$t2`** to the value in **`$zero`** (which is always 0) and stores the result in **`$t1`**.
    - This is effectively a copy operation.

Pseudo Instructions

Examples from the Table: `not $t1, $t2`

- Meaning: Perform a bitwise logical NOT on the value in **`$t2`** (flip all bits) and store the result in **`$t1`**.
- Implementation: **`nor $t1, $t2, $zero`**
- How it works: NOR is “NOT OR”.
  - The operation A NOR 0 is equivalent to NOT A.
  - This is a clever way to achieve a NOT operation.

Pseudo Instructions

Examples from the Table: `neg $t1, $t2`

- Meaning: Negate the value in `$t2` (using two's complement) and store the result in `$t1`.
- Implementation: `sub $t1, $zero, $t2`
- How it works: It subtracts `$t2` from 0, which is the definition of arithmetic negation.

Pseudo Instructions

Examples from the Table: `li $t1, -5`

- Load Immediate
- Meaning: Load the small constant -5 into **register** `$t1`.
- Implementation: **`addiu $t1, $zero, -5`**
- How it works: It adds the sign-extended 16-bit value -5 to **`$zero`** and stores it in **`$t1`**.

Pseudo Instructions

Examples from the Table: `li $t1, 0xabcd1234`

- Load Immediate with a 32-bit constant
- This is the direct application of the concept for a 32-bit constant.
- Meaning: Load the large 32-bit constant into `$t1`.
- Implementation:

```
lui $t1, 0xabcd
ori $t1, $t1, 0x1234
```

	Upper 16 bits	Lower 16 bits
# $t1	0xabcd	0x0000
# $t1	0xabcd	0x1234



Key Points: I-Type Instructions & Datapath

- **R-Type datapath: Fetch, Decode, Execute, Write-Back**
  - Instruction fetched, decoded, ALU computes, result stored
- **I-Type format: opcode | rs | rt | 16-bit immediate**
  - Supports constants, memory addressing, conditional branches
- **I-Type ALU: addi, addiu, andi, ori, slti, lui**
  - lui + ori builds 32-bit constants from two 16-bit halves
- **Pseudo-instructions: move, not, neg, li (assembler translates)**
  - li with large constant becomes lui + ori instruction pair

References

References

Reference Materials

- William Stallings, “Computer Organization and Architecture Designing for Performance,” Tenth Edition, *Pearson*, 2016
- Aiman El-Maleh, “Computer Organization: Lectures,” https://faculty.kfupm.edu.sa, accessed August 26, 2025
- CS2100: Computer Organization, https://www.comp.nus.edu.sg/~adi-yoga/CS2100/ch08c2/

Mahidol University